



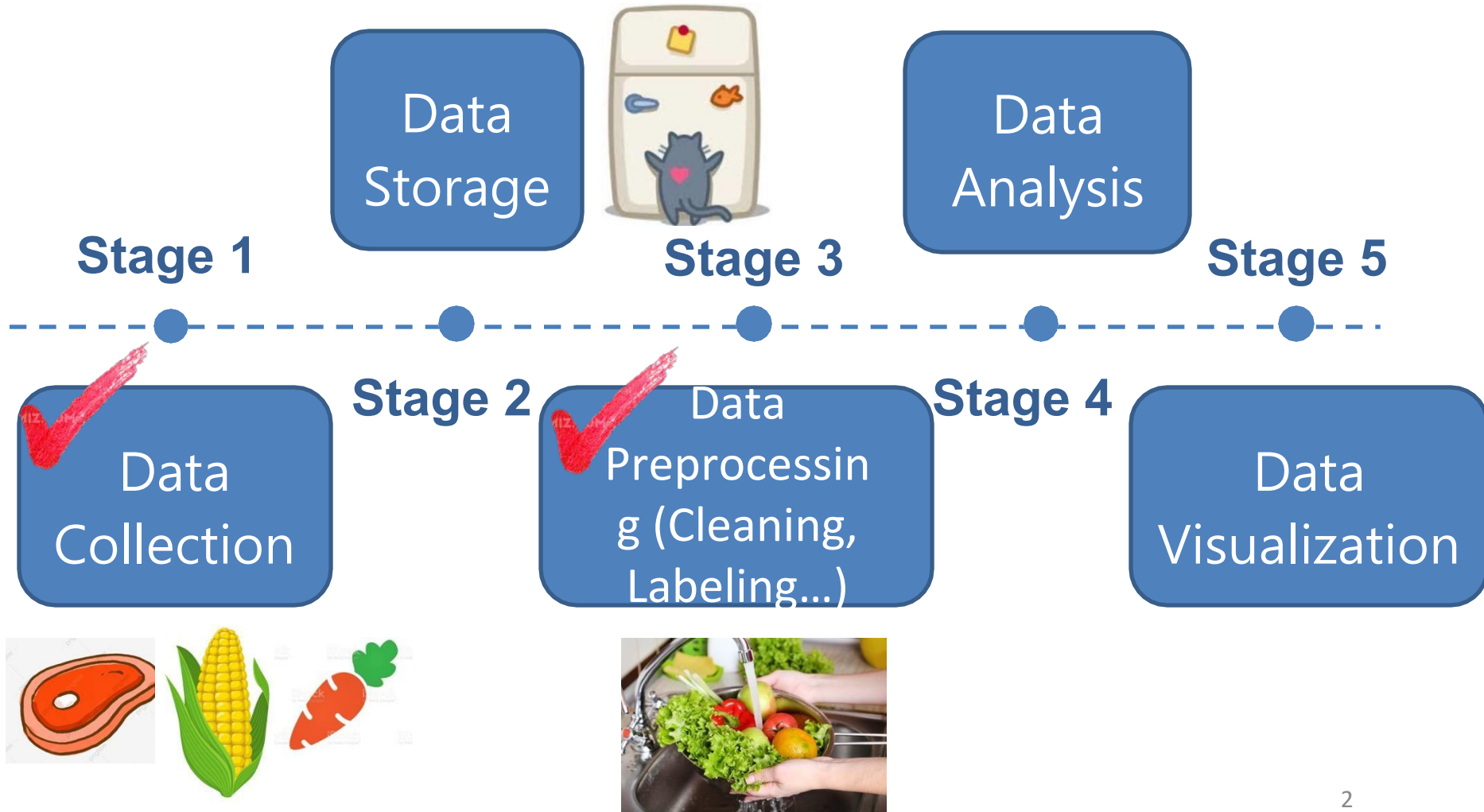
Data Preprocessing

Prof. Chia-Yu Lin
National Central University

2026 Spring

Have Interest in Data Analytics?

Mission : Finish a pork carrot soup



Outline

- Introduction to Data Analytics Libraries
- Basic Concepts of Numpy
- Basic Concepts of Pandas
- Data preprocessing

東京大學資料科學家養成全書：
使用Python動手學習資料分析

- Reference book :



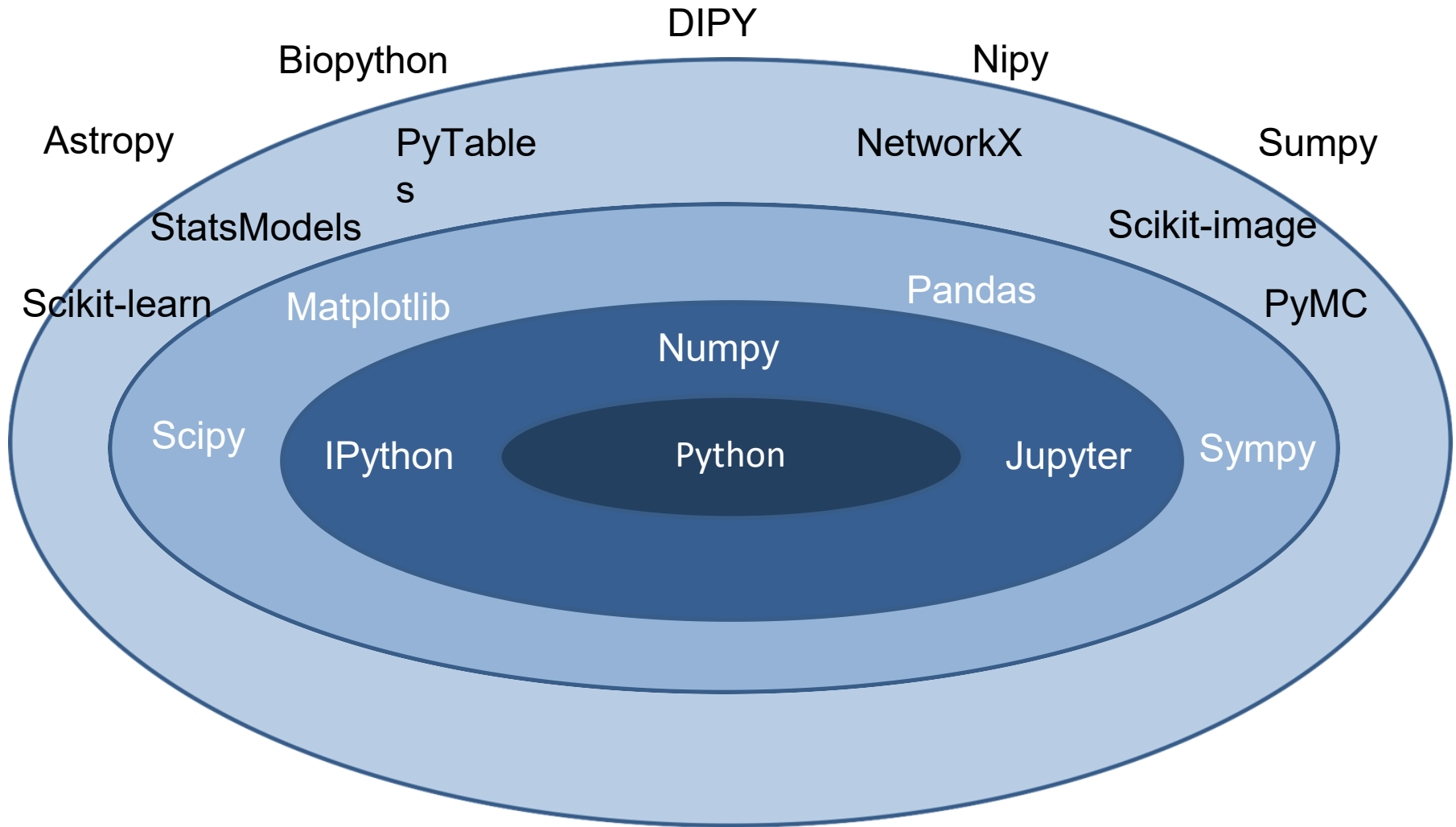


Data Analysis Libraries

- There are four commonly used libraries
- Numpy:
 - A library for basic array and numerical operations.
 - Besides, advanced and complex computation, the computation speed of numpy is faster than normal operations of Python.
- Scipy:
 - A library that further enhance the functionality of Numpy.
 - Can perform statistical and signal operations.
- Pandas
 - A library for processing various data in the form of DataFrames.
- Matplotlib
 - A library for data visualization



Libraries Related to Data Analysis





Import Library

- Using “import”

- Import the Numpy library with the name "np"
- “np.function name” to use the function of Numpy

```
import numpy as np
```

- Using “from”

- Originally, we need to use function by “module.function.function...” to use function
- We can use “from” to omit
 - np.random.function = random.function

```
from numpy import random
```

Magic Command



- Some modules in Jupyter Environment set “Magic Command”
- Magic command **starts with %**, which is a command to perform various environmental operations in the Jupyter environment.
- *%precision*
 - Numpy extension command. When displaying data, you can specify the number of digits after the decimal point to be displayed
- *%matplotlib*
 - The extension command of Matplotlib can specify the display method of the chart.
 - If you use “**inline**”, you can directly display the chart at this position. If you do not specify *%matplotlib*, it will be displayed in another window.

Outline



- Introduction to Data Analytics Modules
- **Basic Concepts of Numpy**
- Basic Concepts of Pandas
- Data preprocessing



Numpy

- NumPy is a Python library used for working with arrays. It also has functions for working in domain of linear algebra, fourier transform, and matrices.
- Numpy is written by C instead of Python. Calculation is fast.

Install Numpy



- `pip install numpy`



Array Operation

- Array

```
#製作陣列
```

```
import numpy as np
```

```
data = np.array([9,2,3,4,10,6,7,8,1,5])
```

```
data
```

```
array([ 9,  2,  3,  4, 10,  6,  7,  8,  1,  5])
```

Data Type (1/3)



- When using Numpy to process data, there is “data type” to enable high-speed calculation and to ensure the precision of the calculation.
- Integer, floating point are data types.

int(Signed Integer) uint(Unsigned Integer)

Data Type	Description
int8	8 bits signed integer
int16	16 bits signed integer
int32	32 bits signed integer
int64	64 bits signed integer

Data Type	Description
uint8	8 bits unsigned integer
uint16	16 bits unsigned integer
uint32	32 bits unsigned integer
uint64	64 bits unsigned integer

Data Type (2/3)



float(Floating point)

Data Type	Description
float16	16 bits floating point
float32	32 bits floating point
float64	64 bits floating point

bool(Boolean)

Data Type	Description
bool	Show True or False



Data Type (3/3)

- If you want to query the data type, you can specify `".dtype"` after the variable.

```
#製作陣列
import numpy as np
data = np.array([9,2,3,4,10,6,7,8,1,5])
data.dtype

dtype('int64')
```

Dimension and Number of Components



- Dimension: `ndim`
- Number of elements: `size`

```
1 import numpy as np
2 data = np.array([9,2,3,4,10,6,7,8,1,5])
3 print('Dimension:',data.ndim)
4 print('Number of elements:',data.size)
```

Dimension: 1

Number of elements: 10



Calculate All Elements

- In Python, you have to use "for loop" to calculate all elements.
- Numpy can calculate all elements without for loop.

```
1 %precision 3
```

```
'%.3f'
```

```
1 import numpy as np
2 data = np.array([9,2,3,4,10,6,7,8,1,5])
3 print('data*2',data*2)
4 print('Multiplication:',np.array([1,2,3,4,5,6,7,8,9,9])*np.array([10,9,8,7,6,5,4,3,2,1]))
5 print('Square:',data**2)
6 print('Division:',np.array([1,2,3,4,5,6,7,8,9,9])/np.array([10,9,8,7,6,5,4,3,2,1]))
```

```
data*2 [18  4  6  8 20 12 14 16  2 10]
Multiplication: [10 18 24 28 30 30 28 24 18  9]
Square: [ 81  4  9 16 100 36 49 64  1 25]
Division: [0.1  0.222 0.375 0.571 0.833 1.2  1.75  2.667 4.5  9.  ]
```




Sort

- Use `sort`. The default is from small to large.
- If you want to show from large to small, you can use “`data[::-1].sort()`”

```
1 import numpy as np
2 data = np.array([9,2,3,4,10,6,7,8,1,5])
3 print('Original data:',data)
4 data.sort()
5 print('From small to large:',data)
6 data[::-1].sort()
7 print('From large to small:',data)
```

```
Original data: [ 9  2  3  4 10  6  7  8  1  5]
From small to large: [ 1  2  3  4  5  6  7  8  9 10]
From large to small: [10  9  8  7  6  5  4  3  2  1]
```

- <Review>
- Q1:What is [n:m:s] in Python ?
 - (1)
- If n and m are omitted, what does it mean?
 - (2)
- What does s=-1 represent ?
 - (3)



Min 、 Max 、 Sum 、 Cumulative Sum

- Array of Numpy can call:
- **min**: minimum
- **max**: maximum
- **sum**: summation
- **cumsum**: cumulative sum
 - For sequence a, b, c
 - First value: a
 - Second value: a+b
 - Third value: a+b+c

```
1 import numpy as np
2 data = np.array([9,2,3,4,10,6,7,8,1,5])
3 print('Minumum:',data.min())
4 print('Maximum:',data.max())
5 print('Summation:',data.sum())
6 print('Cumulative Sum:',data.cumsum())
7 print('Cumulative Sum/Sum:',data.cumsum()/data.sum())
```

```
Minumum: 1
Maximum: 10
Summation: 55
Cumulative Sum: [ 9 11 14 18 28 34 41 49 50 55]
Cumulative Sum/Sum: [0.164 0.2    0.255 0.327 0.509 0.618 0.745 0.891 0.909 1.    ]
```



Random (1/2)

- When we analyze data, we will use “random function” to sperate data or make data in different distribution.
- There is random function in Python, but we usually use random function in Numpy during analyzing data.
- Random function generates random value based on mathematics formula.
- The initial value of random function is called “seed.”
- Although it is not necessary to specify a seed during the operation, specifying the same seed can ensure obtain the same sequence of random numbers in each operation.

```
1 import numpy.random as random
2
3 #Specify random seed
4 random.seed(0)
5 #Generate 10 random numbers based on normal distribution (mean is 0, std is 1)
6 rnd_data = random.randn(10)
7
8 print('Array contains 10 random numbers:', rnd_data)
```

```
Array contains 10 random numbers: [ 1.764  0.4    0.979  2.241  1.868 -0.977  0.95  -0.151 -
0.103  0.411]
```

Random (2/2)



function	Definition
rand	Uniform distribution · $0.0 < x < 1.0$
random_sample	Uniform distribution · $0.0 < x < 1.0$ (The generating method is different from "rand")
randint	Uniform distribution · Integers without range.
randn	Normal distribution. Random numbers whose mean is 0 and std is 1.
normal	Normal distribution. Random numbers with arbitrary mean and std.
binomial	Random numbers from binomial distribution.
beta	Random numbers from beta distribution.
gamma	Random numbers from gamma distribution.
chisquare	Random numbers from chi-squared distribution.



Retrieve Data Randomly

- `random.choice`
- There are 2 arguments and 1 parameter.
 - The first argument : array you want to retrieve data.
 - The second argument : Number of data you want to retrieve.
 - Parameter : `replace`
 - If `replace` is `True` or without specifying, it means data can be retrieved repeatedly.
 - If `replace` is `False`, it means data can not be retrieved repeatedly.

```
1 data = np.array([9,2,3,4,10,6,7,8,1,5])
2
3 # Retrieve data randomly
4 # Retrieve 10 data. Allow repeated data.
5 print(random.choice(data, 10))
6 # Retrieve 10 data. Not Allow repeated data.
7 print(random.choice(data, 10, replace = False))
```

```
[ 7  8  8  1  2  6  5  1  5 10]
[10  2  7  8  3  1  6  5  9  4]
```



Matrix

- We can use Numpy to do matrix operation.
- “arrange” function can generate specified consecutive integers.
 - `arange(9)` can generate integer 0~8.
- Retrieve row or column from matrix, we can use “[range of row, range of column].”

```
1 import numpy as np
2
3 array1 = np.arange(9)
4 print(array1)
5 array2 = np.arange(9).reshape(3,3)
6 print(array2)
7 #The first row
8 print('The first row',array2[0,:])
9 #The first column
10 print('The first column',array2[:,0])
```

```
[0 1 2 3 4 5 6 7 8]
```

```
[[0 1 2]
```

```
 [3 4 5]
```

```
 [6 7 8]]
```

```
The first row [0 1 2]
```

```
The first column [0 3 6]
```



Matrix Multiplication

- “dot function” is matrix multiplication.
- If use “*” , it is element-wise product. (The respective elements are multiplied.)

```
1 import numpy as np
2
3 array1 = np.arange(9).reshape(3,3)
4 array2 = np.arange(9,18).reshape(3,3)
5 print('array1:',array1)
6 print('array2:',array2)
7 print('Matrix Mutiplication: ',np.dot(array1,array2))
8 print('Element-wise Product: ',array1*array2)
```

```
array1: [[0 1 2]
         [3 4 5]
         [6 7 8]]
array2: [[ 9 10 11]
         [12 13 14]
         [15 16 17]]
Matrix Mutiplication: [[ 42  45  48]
                       [150 162 174]
                       [258 279 300]]
Element-wise Product: [[ 0 10 22]
                       [36 52 70]
                       [90 112 136]]
```

Matrix Whose Elements Are 0 Or 1



- *np.zeros*: Matrix whose elements are 0
- *np.ones*: Matrix whose elements are 1
- *dtype*: Specify data type

```
import numpy as np

print(np.zeros((2, 3), dtype = np.int64))
print(np.ones((2, 3), dtype = np.float64))
```

```
[[0 0 0]
 [0 0 0]]
```

```
[[1. 1. 1.]
 [1. 1. 1.]]
```




Outline

- Introduction to Data Analytics Modules
- Basic Concepts of Numpy
- **Basic Concepts of Pandas**
- Data preprocessing

Pandas



- In Python, Pandas is a library to preprocess data before building model.
- Pandas can flexibly process all kinds of data, and perform operations such as table calculation, data extraction, and search.
- EX:
 - Find rows from data that match certain criteria
 - Set a benchmark to calculate the average of each
 - Merge data

Install Pandas



- `pip install pandas`



Import Pandas

- Import Pandas

```
import pandas as pd
```

- Import **Series** module, which can process **one dimension array**.
- Import **DataFrame** module, which can process **two dimension array**.

```
from pandas import Series, DataFrame
```



Series (1/3)

- A pandas Series is a one-dimensional labelled data structure which can hold data such as strings, integers and even other Python objects.
- The basis of Series of Pandas is the array of Numpy.

Series

```
: sample_pandas_data = pd.Series([0,10,20,30,40,50,60,70,80,90])  
print(sample_pandas_data)
```

0	0
1	10
2	20
3	30
4	40
5	50
6	60
7	70
8	80
9	90

Elements

dtype: int64

Data Type

Series (2/3)



- Get data : `Series.values`
- Get index :
`Series.index`

```
1 print('Data:', sample_pandas_data.values)
2 print('Index:', sample_pandas_data.index)
```

Data: [0 10 20 30 40 50 60 70 80 90]

Index: RangeIndex(start=0, stop=10, step=1)



Series (3/3)

- Specify index in Series

Specify index in Series

```
sample_pandas_index_data = pd.Series(  
    [0, 10, 20, 30, 40, 50, 60, 70, 80, 90],  
    index=['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j'])  
  
print(sample_pandas_index_data)
```

```
a      0  
b     10  
c     20  
d     30  
e     40  
f     50  
g     60  
h     70  
i     80  
j     90  
dtype: int64
```

```
1 print('Data:', sample_pandas_index_data.values)  
2 print('Index:', sample_pandas_index_data.index)
```

```
Data: [ 0 10 20 30 40 50 60 70 80 90]
```

```
Index: Index(['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j'], dtype='object')
```



DataFrame (1/2)

- A Pandas DataFrame is a 2 dimensional data structure, like a 2 dimensional array, or a table with rows and columns.
- We can set difference dtype (data type) for each column.

DataFrame

```
attri_data1 = {'ID': ['100', '101', '102', '103', '104'],  
               'City': ['Taipei', 'Taipei', 'Hsinchu', 'Tainan', 'Kaohsiung'],  
               'Birth_year': [1990, 1989, 1992, 1997, 1982],  
               'Name': ['Sally', 'Hanks', 'Hannah', 'Kitty', 'Steve']}
```

```
attri_data_frame1 = DataFrame(attri_data1)
```

```
print(attri_data_frame1)
```

Element
Index

	ID	City	Birth_year	Name
0	100	Taipei	1990	Sally
1	101	Taipei	1989	Hanks
2	102	Hsinchu	1992	Hannah
3	103	Tainan	1997	Kitty
4	104	Kaohsiung	1982	Steve



DataFrame (2/2)

- DataFrame, like Series, can change the index value and set the text as the index value

Specify Index in DataFrame

```
: attri_data_frame_index1 = DataFrame(attri_data1, index=['a', 'b', 'c', 'd', 'e'])  
print(attri_data_frame_index1)
```

Element
Index

	ID	City	Birth_year	Name
a	100	Taipei	1990	Sally
b	101	Taipei	1989	Hanks
c	102	Hsinchu	1992	Hannah
d	103	Tainan	1997	Kitty
e	104	Kaohsiung	1982	Steve



In Jupyter

- Previously, we use “print” to show Series or DataFrame.
- In Jupyter, the Series or DataFrame can be automatically recognized.
- Therefore, we can directly input Series or DataFrame.

```
In [22]: attri_data_frame_index1
```

```
Out[22]:
```

	ID	City	Birth_year	Name
a	100	Taipei	1990	Sally
b	101	Taipei	1989	Hanks
c	102	Hsinchu	1992	Hannah
d	103	Tainan	1997	Kitty
e	104	Kaohsiung	1982	Steve



Exchange Row and Column of DataFrame

- Exchange row and column, like the transpose in matrix.

Transpose

In [23]: `attri_data_frame1.T`

Out[23]:

	0	1	2	3	4
ID	100	101	102	103	104
City	Taipei	Taipei	Hsinchu	Tainan	Kaohsiung
Birth_year	1990	1989	1992	1997	1982
Name	Sally	Hanks	Hannah	Kitty	Steve

Extracted Specific Column (1/2)



- Extracted the specified one column: Specify the column name directly.

Get One Column

```
In [15]: attri_data_frame1.Birth_year
```

```
Out[15]: 0      1990  
         1      1989  
         2      1992  
         3      1997  
         4      1982  
         Name: Birth_year, dtype: int64
```

Extracted Specific Column(2/2)



- Extracted the specified many columns: Use “list” of Python to specify.

Get Many Column

```
In [17]: attri_data_frame1[['ID', 'Birth_year']]
```

Out[17]:

	ID	Birth_year
0	100	1990
1	101	1989
2	102	1992
3	103	1997
4	104	1982

Extract column.
Without “.”



Extract Data (1/3)

- For DataFrame objects, you can only get data that meets certain conditions, or combine multiple data => Like a filter.

Filter

```
In [26]: attri_data_frame1[attri_data_frame1['City'] == 'Taipei']
```

```
Out[26]:
```

	ID	City	Birth_year	Name
0	100	Taipei	1990	Sally
1	101	Taipei	1989	Hanks



Extract Data (2/3)

- Extract “City Column” and compare “City Column” to “Taipei.” The result shows True or False.

```
In [28]: attri_data_frame1['City'] == 'Taipei'
```

```
Out[28]: 0      True
          1      True
          2     False
          3     False
          4     False
          Name: City, dtype: bool
```



Extract Data (3/3)

- If you want to specify multiple conditions, you can use

“isin(list).”

Filter for Many Requirements

```
attri_data_frame1[attri_data_frame1['City'].isin(['Taipei', 'Tainan'])]
```

	ID	City	Birth_year	Name
0	100	Taipei	1990	Sally
1	101	Taipei	1989	Hanks
3	103	Tainan	1997	Kitty

Drop Column or Row in DataFrame

(1/4)



- To remove specific column or row, we can use “drop.”
- Use “axis parameter” to specify column or row.
 - “axis=0” is row
 - “axis=1” is column

Drop Column or Row in DataFrame

(2/4)



- Drop Column :

Drop Column

```
attri_data_frame1.drop(['Birth_year'], axis = 1)
```

	ID	City	Name
0	100	Taipei	Sally
1	101	Taipei	Hanks
2	102	Hsinchu	Hannah
3	103	Tainan	Kitty
4	104	Kaohsiung	Steve

Drop Column or Row in DataFrame

(3/4)



- When we execute “`attri_data_frame1.drop(['Birth_year'], axis = 1),`” the column is not deleted “in data.”
- If you want to delete the column “in data,” you have to use “assign.”

Drop Column or Row in DataFrame

(4/4)



```
attri_data_frame1 = DataFrame(attri_data1)
print(attri_data_frame1)
```

```
attri_data_frame1.drop(['Birth_year'], axis = 1)
print(attri_data_frame1)
```

```
attri_data_frame1 = attri_data_frame1.drop(['Birth_year'], axis = 1)
print(attri_data_frame1)
```

	ID	City	Birth_year	Name
0	100	Taipei	1990	Sally
1	101	Taipei	1989	Hanks
2	102	Hsinchu	1992	Hannah
3	103	Tainan	1997	Kitty
4	104	Kaohsiung	1982	Steve

	ID	City	Name
0	100	Taipei	Sally
1	101	Taipei	Hanks
2	102	Hsinchu	Hannah
3	103	Tainan	Kitty
4	104	Kaohsiung	Steve

Merge Data in DataFrame (1/3)



- The objects of DataFrame can be merged.
- Data are from difference source. We need to merge data to analyze.
- We can use “merge.”

Merge Data in DataFrame (2/3)



- Merge dataframe “attri_data_frame1” and dataframe “ attri_data_frame2”
- The common column of these two dataframes is “ID.”
- Choose the data with same ID to merge.

	ID	City	Birth_year	Name
0	100	Taipei	1990	Sally
1	101	Taipei	1989	Hanks
2	102	Hsinchu	1992	Hannah
3	103	Tainan	1997	Kitty
4	104	Kaohsiung	1982	Steve

	ID	Math	English	Sex
0	100	50	90	M
1	101	43	30	F
2	102	33	20	F
3	105	76	50	M
4	107	98	30	M

Merge Data in DataFrame (3/3)



```
In [55]: pd.merge(attri_data_frame1, attri_data_frame2)
```

Out[55]:

	ID	City	Birth_year	Name	Math	English	Sex
0	100	Taipei	1990	Sally	50	90	M
1	101	Taipei	1989	Hanks	43	30	F
2	102	Hsinchu	1992	Hannah	33	20	F

Statistics (1/2)



- We can do statistics in DataFrame.
- Use “groupby” to do statistics based on specific condition/behavior.

Statistics (2/2)



- Use the Gender row as a benchmark to calculate the average score for math grades.

Statistics

```
In [56]: attri_data_frame2.groupby('Sex')['Math'].mean()
```

```
Out[56]: Sex
F      38.000000
M      74.666667
Name: Math, dtype: float64
```

Sort (1/3)



- For objects of Series and DataFrame, we can do sorting.
- Sort by index
- Sort by value

```
: attri_data2 = {'ID': ['100', '101', '102', '103', '104'],  
                'City': ['Taipei', 'Taipei', 'Hsinchu', 'Tainan', 'Kaohsiung'],  
                'Birth_year': [1990, 1989, 1992, 1997, 1982],  
                'Name': ['Sally', 'Hanks', 'Hannah', 'Kitty', 'Steve']}  
attri_data_frame_index2 = DataFrame(attri_data2, index=['e', 'b', 'a', 'd', 'c'])  
attri_data_frame_index2
```

```
:
```

	ID	City	Birth_year	Name
e	100	Taipei	1990	Sally
b	101	Taipei	1989	Hanks
a	102	Hsinchu	1992	Hannah
d	103	Tainan	1997	Kitty
c	104	Kaohsiung	1982	Steve

Sort (2/3)



- Sort by index

```
: #sort by index  
attri_data_frame_index2.sort_index()  
:
```

	ID	City	Birth_year	Name
a	102	Hsinchu	1992	Hannah
b	101	Taipei	1989	Hanks
c	104	Kaohsiung	1982	Steve
d	103	Tainan	1997	Kitty
e	100	Taipei	1990	Sally

Sort (3/3)



- Sort by value
- Use “value of Birth_year column” to sort.

```
#sort by values
```

```
attri_data_frame_index2.Birth_year.sort_values()
```

```
c      1982
```

```
b      1989
```

```
e      1990
```

```
a      1992
```

```
d      1997
```

```
Name: Birth_year, dtype: int64
```

Check nan (null) (1/4)



- Sometime, there is missing information in data.
- If we directly calculate average, we cannot get correct result
- Missing information should be deleted.

Check nan (null) (2/4)



- Compare eligible data
 - Check whether the data contains “Taipei”

```
1 #Check whether the data contains "Taipei"  
2 attri_data_frame_index2.isin(['Taipei'])
```

	ID	City	Birth_year	Name
e	False	True	False	False
b	False	True	False	False
a	False	False	False	False
d	False	False	False	False
c	False	False	False	False

Check nan (null) (3/4)

- Use “**isnull**” to check whether the value is “nan.”

```
1 # Check nan (null)
2 # Set Name as nan
3 import numpy as np
4 attri_data_frame_index2['Name'] = np.nan
5
6 #Check whether the value is "nan."
7 attri_data_frame_index2.isnull()
```

	ID	City	Birth_year	Name
e	100	Taipei	1990	NaN
b	101	Taipei	1989	NaN
a	102	Hsinchu	1992	NaN
d	103	Tainan	1997	NaN
c	104	Kaohsiung	1982	NaN



	ID	City	Birth_year	Name
e	False	False	False	True
b	False	False	False	True
a	False	False	False	True
d	False	False	False	True
c	False	False	False	True

Check nan (null) (4/4)



- Calculate the number of nan

```
attri_data_frame_index2.isnull().sum()
```

ID	0
City	0
Birth_year	0
Name	5
dtype: int64	

There are 5 True in "Name" column.

Outline



- Introduction to Data Analytics Modules
- Basic Concepts of Numpy
- Basic Concepts of Pandas
- Data preprocessing

The Importance of Data Preprocessing



- If there is some soil and dirt in your carrot pork soup, how do you feel?
- **Cleaning** is absolutely indispensable step in processing food materials.
- Same as data. Data needs preprocessing.
- **Wrong data** cannot produce correct result even though you have powerful analytical method.

Data Preprocessing

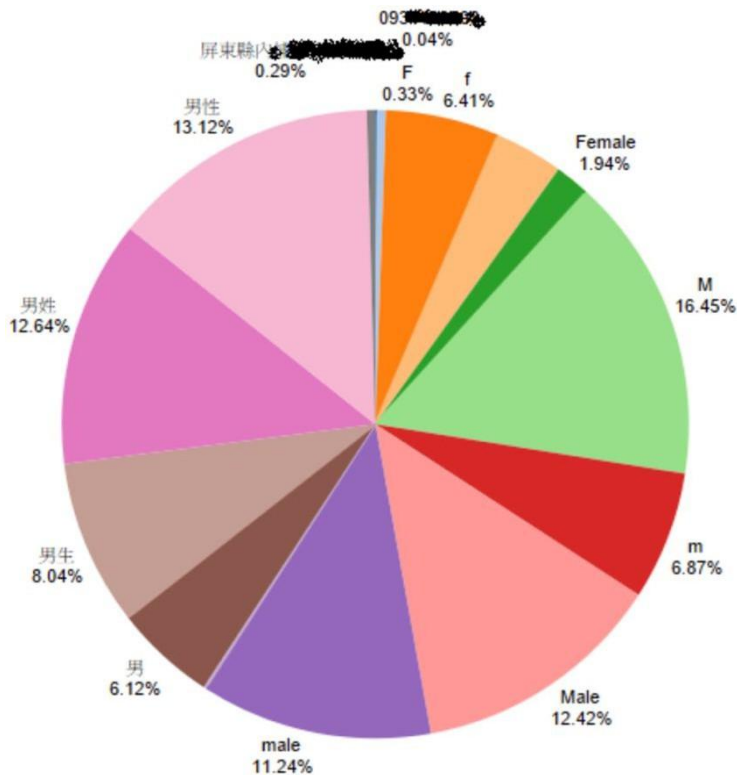


1. Data Cleaning (資料清洗)
2. Impute missing value (資料補值)
3. Data Labeling (資料標註)

Data Cleaning (1/3)



- If some data of a company contains boy, girl, male, female, Male, Female, M, F, ...etc., many columns are duplicate.



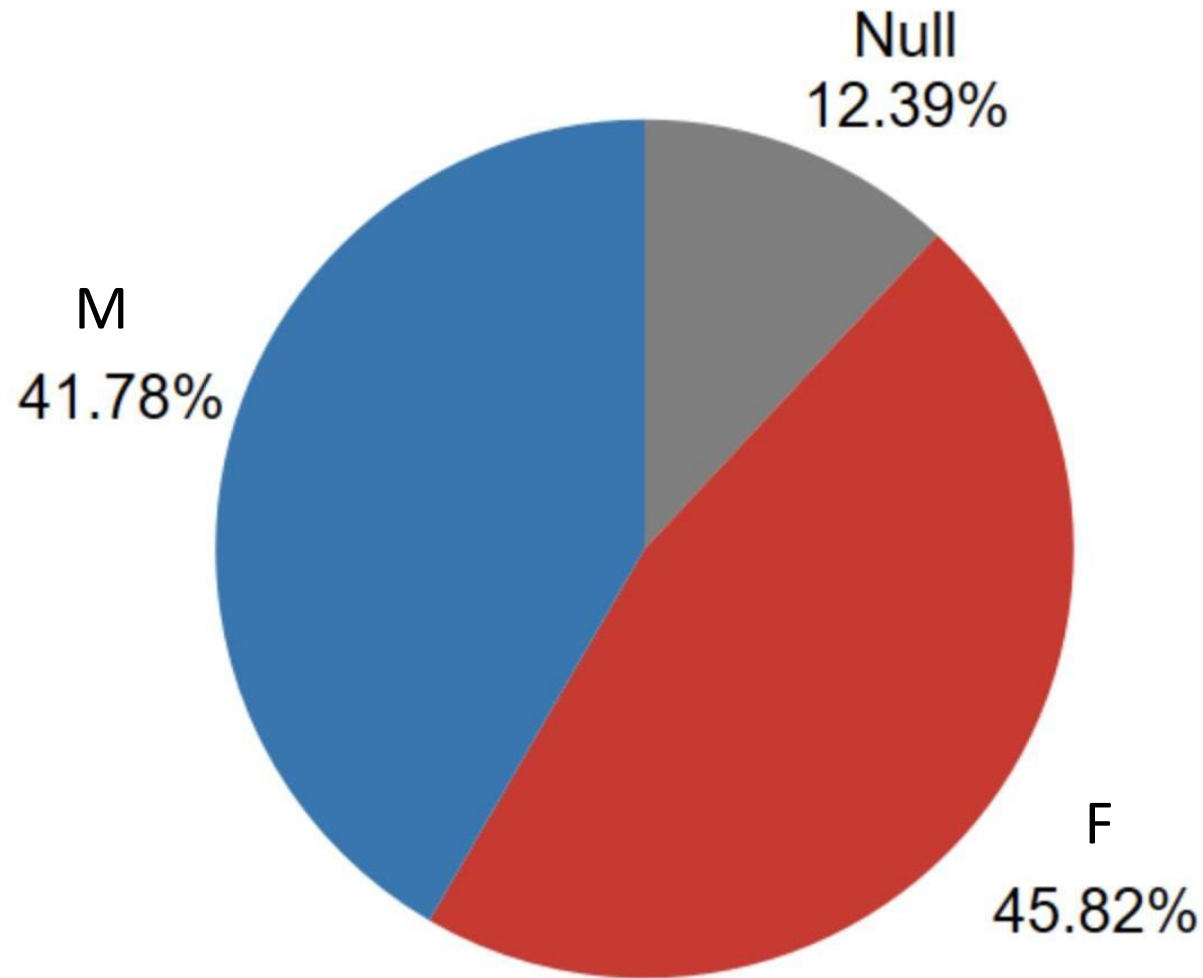
Pie charts are divided into various blocks!
Many blocks are duplicate.

Data Cleaning (2/3)



- First, decide the format of data.
 - “Boy, Girl”
 - “Male, Female”
 - “M, F”
- If the format is set as “M, F”, we need to start convergence process. Change all different format of male and female to “M, F”.
- Finally, if there are some value does not represent male and female, such as cell phone number, address. We should change the value to “null.”

Data Cleaning (3/3)



Data Cleaning : Outlier



- Outlier :
- There is no uniform definition. The outliers are judged by data analysts or decision maker.
- Methods to judge outlier:
 - Draw boxplots. Treat values above a certain percentage as outliers.
 - Using normal distribution.
 - Map data into a specific space and observe the distance among data.

Data Preprocessing



1. Data Cleaning (資料清洗)
2. Impute missing value (資料補值)
3. Data Labeling (資料標註)

Impute missing value



- Missing value and outlier unavoidable situations when dealing with data.
- There are various reasons for missing value. For example, forgetting to fill the data, system issue.
- For missing value, should it be ignored, or the closest value fill in? ??
 - Different methods will produce great deviations, which may lead to wrong decisions and cause heavy losses. Thus, missing value should be handled with caution.

Example Data



- Assume “NaN(NA)” is missing value.

```
#Data Preparation
import numpy as np
from numpy import nan as NA
import pandas as pd

random.seed(0)
df = pd.DataFrame(np.random.rand(10, 4))

# 設定為NA
df.iloc[1,0] = NA
df.iloc[2:3,2] = NA
df.iloc[5:,3] = NA
print(df)
```

	0	1	2	3
0	0.548814	0.715189	0.602763	0.544883
1	NaN	0.645894	0.437587	0.891773
2	0.963663	0.383442	NaN	0.528895
3	0.568045	0.925597	0.071036	0.087129
4	0.020218	0.832620	0.778157	0.870012
5	0.978618	0.799159	0.461479	NaN
6	0.118274	0.639921	0.143353	NaN
7	0.521848	0.414662	0.264556	NaN
8	0.456150	0.568434	0.018790	NaN
9	0.612096	0.616934	0.943748	NaN

Listwise Deletion (成批刪除)



- Delete the row which has NaN.
- Use “**dropna**,” which is called **list-wise deletion** (成批刪除).

```
df = df.dropna()
```

	0	1	2	3
0	0.548814	0.715189	0.602763	0.544883
1	NaN	0.645894	0.437587	0.891773
2	0.963663	0.383442	NaN	0.528895
3	0.568045	0.925597	0.071036	0.087129
4	0.020218	0.832620	0.778157	0.870012
5	0.978618	0.799159	0.461479	NaN
6	0.118274	0.639921	0.143353	NaN
7	0.521848	0.414662	0.264556	NaN
8	0.456150	0.568434	0.018790	NaN
9	0.612096	0.616934	0.943748	NaN



	0	1	2	3
0	0.548814	0.715189	0.602763	0.544883
3	0.568045	0.925597	0.071036	0.087129
4	0.020218	0.832620	0.778157	0.870012

Pairwise deletion (逐對刪除)



- List-wise Deletion will make data become too little, make the data unusable.
- In pairwise deletion, we can ignore column which has missing value.
- We extract the columns we want and use dropna.

```
df = df[[0,1]].dropna()
```

	0	1	2	3
0	0.548814	0.715189	0.602763	0.544883
1	NaN	0.645894	0.437587	0.891773
2	0.963663	0.383442	NaN	0.528895
3	0.568045	0.925597	0.071036	0.087129
4	0.020218	0.832620	0.778157	0.870012
5	0.978618	0.799159	0.461479	NaN
6	0.118274	0.639921	0.143353	NaN
7	0.521848	0.414662	0.264556	NaN
8	0.456150	0.568434	0.018790	NaN
9	0.612096	0.616934	0.943748	NaN



	0	1
0	0.548814	0.715189
2	0.963663	0.383442
3	0.568045	0.925597
4	0.020218	0.832620
5	0.978618	0.799159
6	0.118274	0.639921
7	0.521848	0.414662
8	0.456150	0.568434
9	0.612096	0.616934

Filling Missing Value: fillna

- Use “fillna(value)” for NaN value.
- EX: If we want to fill 0 for NaN, we can use fillna(0).

```
df = df.fillna(0)
```

	0	1	2	3
0	0.548814	0.715189	0.602763	0.544883
1	NaN	0.645894	0.437587	0.891773
2	0.963663	0.383442	NaN	0.528895
3	0.568045	0.925597	0.071036	0.087129
4	0.020218	0.832620	0.778157	0.870012
5	0.978618	0.799159	0.461479	NaN
6	0.118274	0.639921	0.143353	NaN
7	0.521848	0.414662	0.264556	NaN
8	0.456150	0.568434	0.018790	NaN
9	0.612096	0.616934	0.943748	NaN



	0	1	2	3
0	0.548814	0.715189	0.602763	0.544883
1	0.000000	0.645894	0.437587	0.891773
2	0.963663	0.383442	0.000000	0.528895
3	0.568045	0.925597	0.071036	0.087129
4	0.020218	0.832620	0.778157	0.870012
5	0.978618	0.799159	0.461479	0.000000
6	0.118274	0.639921	0.143353	0.000000
7	0.521848	0.414662	0.264556	0.000000
8	0.456150	0.568434	0.018790	0.000000
9	0.612096	0.616934	0.943748	0.000000

Filling Missing Value: ffill

- We can use “**ffill**” to fill in the value of “previous row.”

```
df = df.fillna(method = 'ffill')
```

	0	1	2	3
0	0.548814	0.715189	0.602763	0.544883
1	NaN	0.645894	0.437587	0.891773
2	0.963663	0.383442	NaN	0.528895
3	0.568045	0.925597	0.071036	0.087129
4	0.020218	0.832620	0.778157	0.870012
5	0.978618	0.799159	0.461479	NaN
6	0.118274	0.639921	0.143353	NaN
7	0.521848	0.414662	0.264556	NaN
8	0.456150	0.568434	0.018790	NaN
9	0.612096	0.616934	0.943748	NaN



	0	1	2	3
0	0.548814	0.715189	0.602763	0.544883
1	0.548814	0.645894	0.437587	0.891773
2	0.963663	0.383442	0.437587	0.528895
3	0.568045	0.925597	0.071036	0.087129
4	0.020218	0.832620	0.778157	0.870012
5	0.978618	0.799159	0.461479	0.870012
6	0.118274	0.639921	0.143353	0.870012
7	0.521848	0.414662	0.264556	0.870012
8	0.456150	0.568434	0.018790	0.870012
9	0.612096	0.616934	0.943748	0.870012



Filling Missing Value: mean

- We can use “mean” to fill the average value of the column.
- Notice: In time series data, if you use this method, you may use future value to compute mean. You should avoid this.

```
print(df.mean())  
df = df.fillna(df.mean())
```

The average value for each column.

```
0    0.531970  
1    0.654185  
2    0.413497  
3    0.584539  
dtype: float64
```

	0	1	2	3
0	0.548814	0.715189	0.602763	0.544883
1	0.531970	0.645894	0.437587	0.891773
2	0.963663	0.383442	0.413497	0.528895
3	0.568045	0.925597	0.071036	0.087129
4	0.020218	0.832620	0.778157	0.870012
5	0.978618	0.799159	0.461479	0.584539
6	0.118274	0.639921	0.143353	0.584539
7	0.521848	0.414662	0.264556	0.584539
8	0.456150	0.568434	0.018790	0.584539
9	0.612096	0.616934	0.943748	0.584539

More fillna



- Besides the methods we introduce, there are more fillna methods.
- Use “`?df.fillna`” to search other methods.
- About missing value, above methods do not work in all cases . When dealing with missing value, you should consider the background and situation of data and choose appropriate fillna method.

Data Preprocessing



1. Data Cleaning (資料清洗)
2. Impute missing value (資料補值)
3. Data Labeling (資料標註)

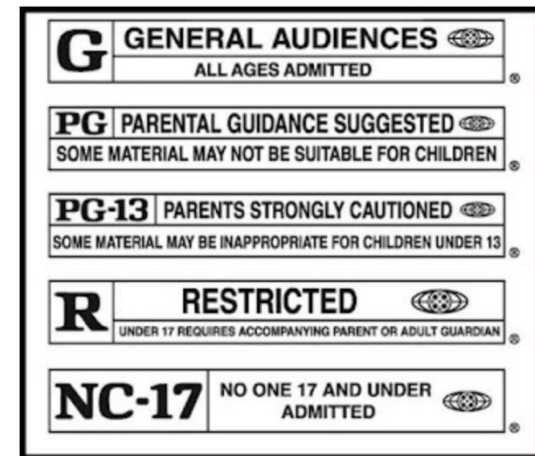
Data Labeling



- Label the features in



- Label the correct answer in data.

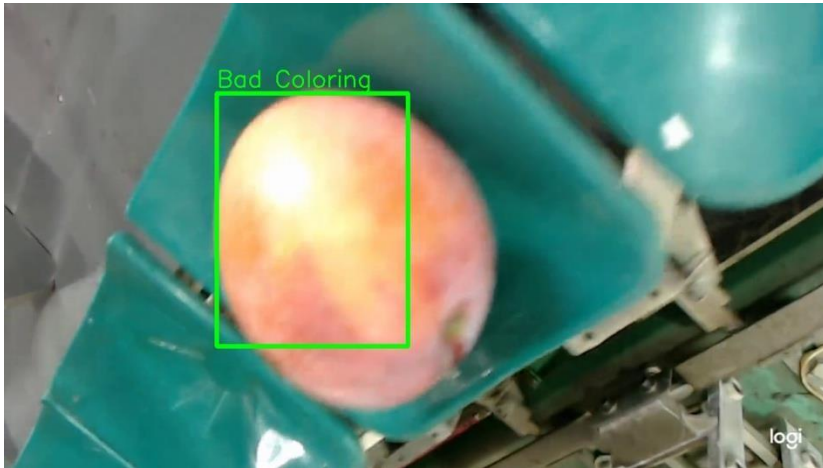


Example of Data Labeling

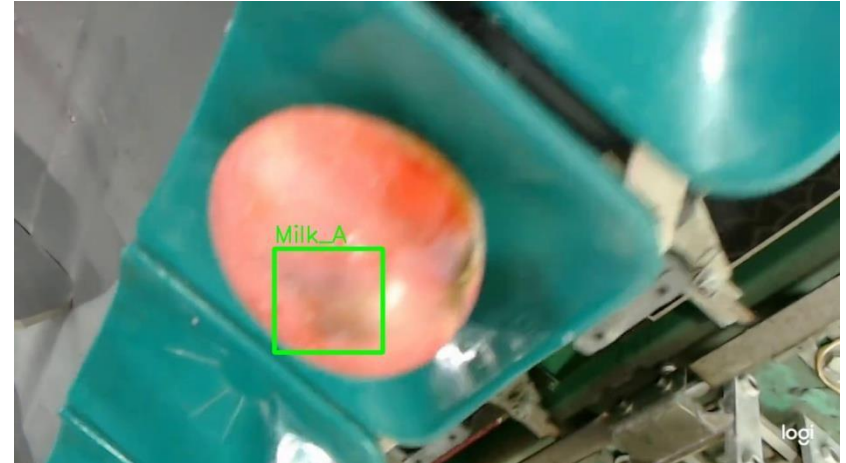


- Label the correct answer in data.

The color is not good (著色不佳)



Milk Residue (乳汁吸附)



Tool of Data Labeling



- Labelbox
- Labellmg
 - Support Yolo



Test



- What are the 3 parts of data preprocessing?

- (1)

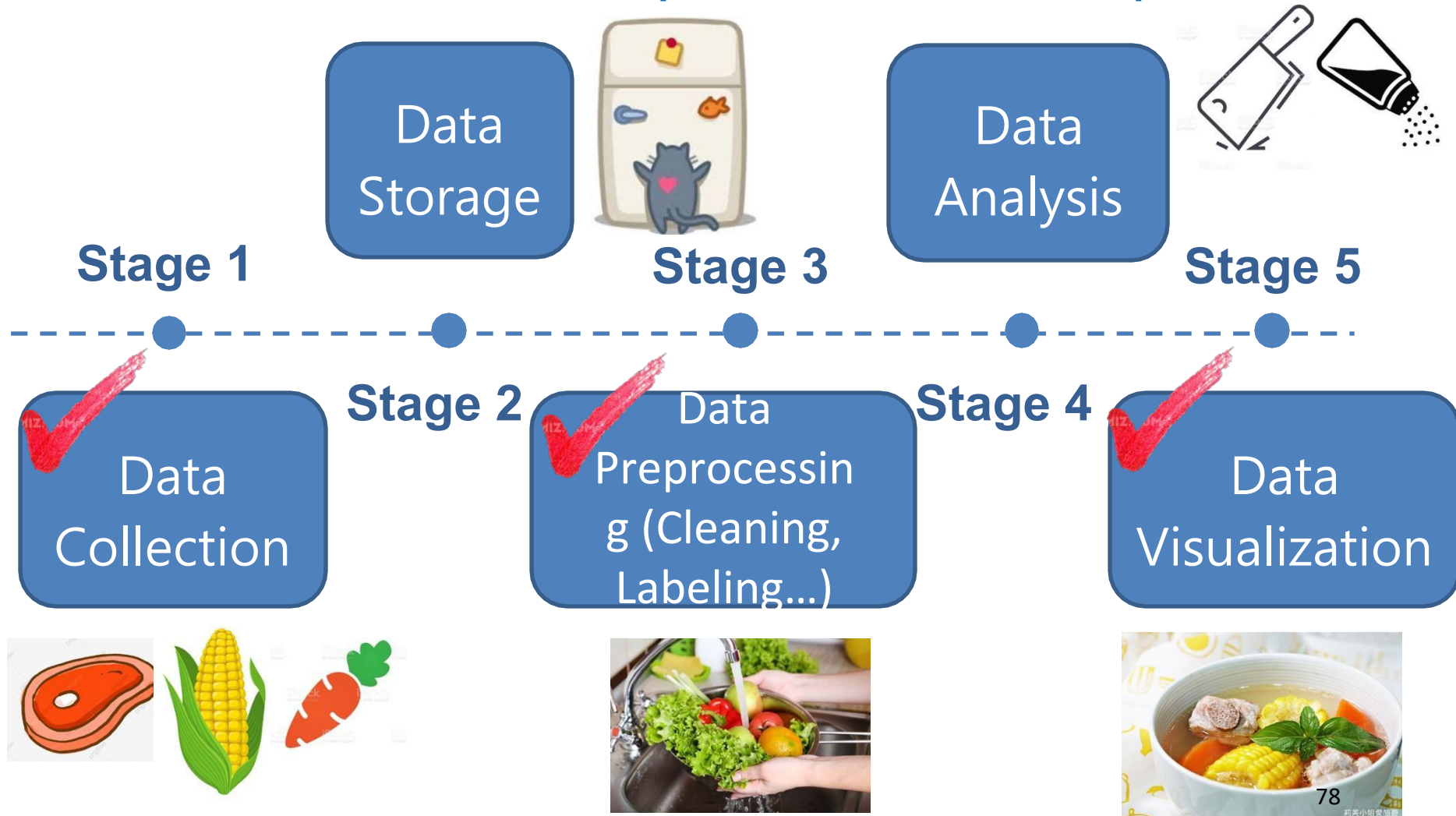
- (2)

- (3)

Have Interest in Data Analytics?



Mission : Finish a pork carrot soup



Outline of Data Visualization



- What is data visualization?
- Basic library for data visualization.
- Advanced library data visualization.
- Stock market data visualization.

Data Visualization



- Before analyzing data.....
 - Sometimes you can't find a message by just observing the numbers.
 - We can obtain some **implicit information** by data visualization.
 - **Enhance data understanding** through charts and infographics.
- After analyzing data.....
 - Analysis results are easily interpreted with graphs.
 - By converting the information into charts, it is **easier** for people to understand.

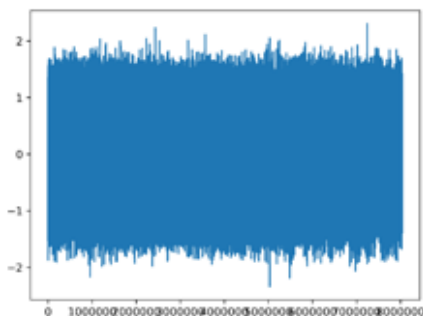
Example of Data Visualization before Analyzing Data



- Visualize data to see whether it conforms your understanding.

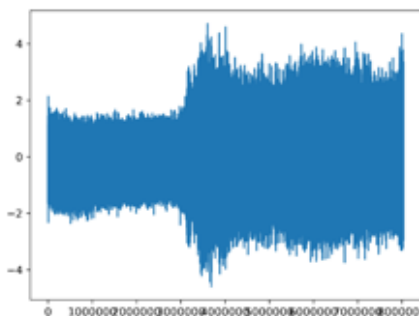
Before anomaly

進入異常前(1)



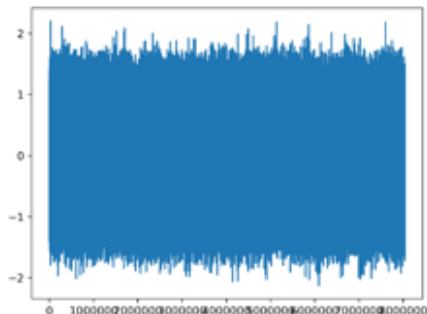
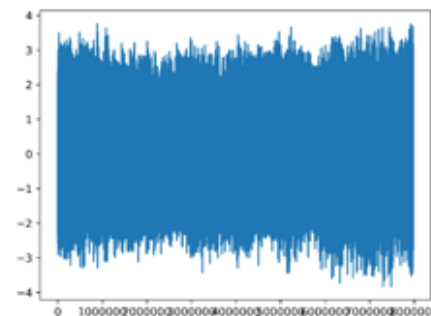
Start to have anomaly

開始進入異常

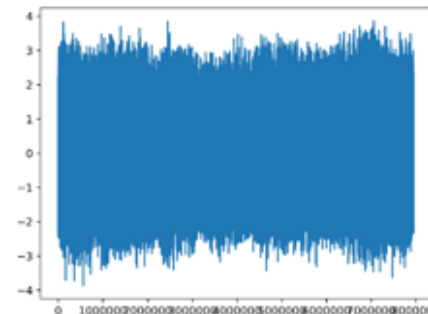


After anomaly

進入異常後(2)



進入異常前(2)



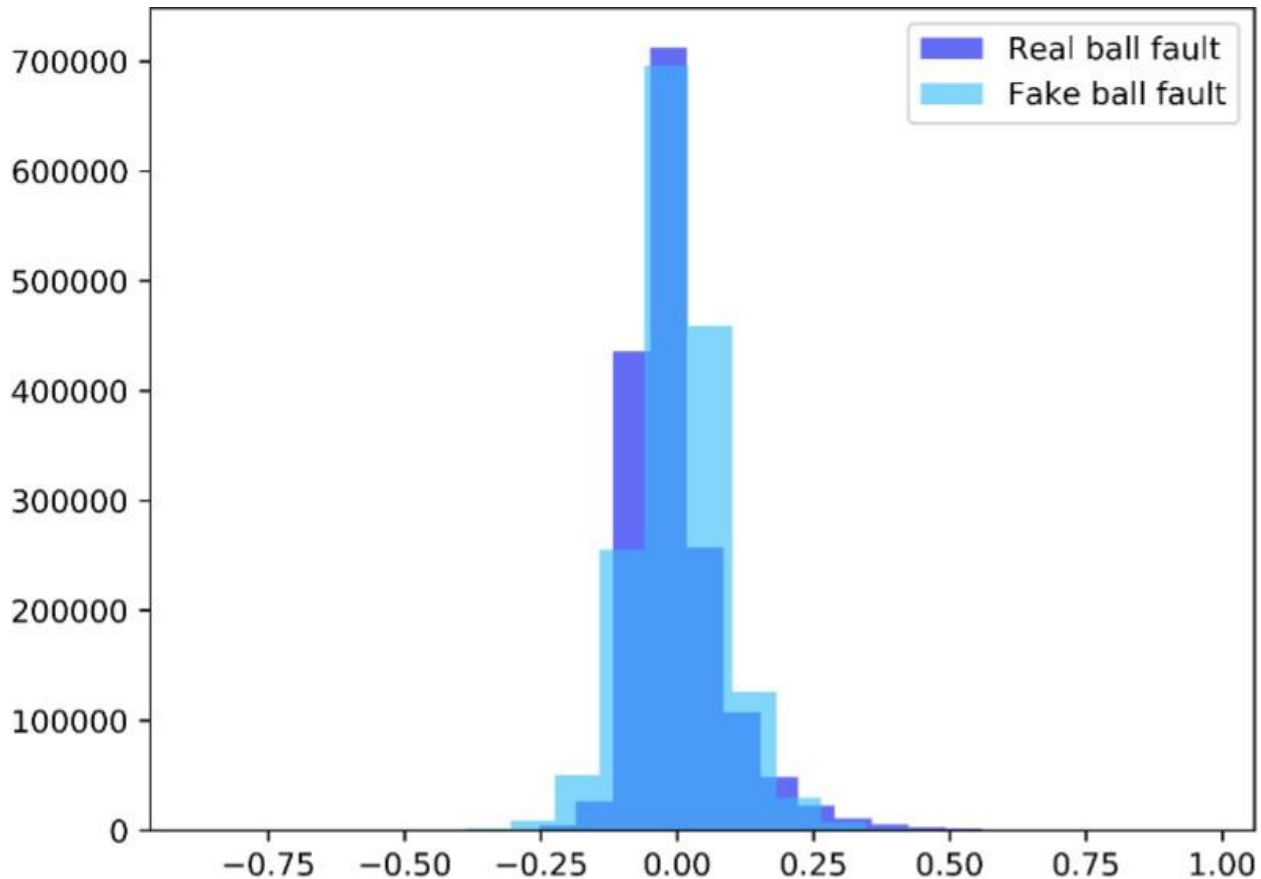
進入異常後(1)

Example of Data Visualization



After Analyzing Data

- The distribution of generated data and real world data.



(a) Distribution of real and fake ball fault.

Outline of Data Visualization



- What is data visualization?
- **Basic library for data visualization.**
- Advanced library data visualization.
- Stock market data visualization.

Library for Data Visualization



- Matplotlib
 - In Matplotlib, most of data visualization function is provided by “`pyplot.function name`”
 - Thus, after importing matplotlib library by “`import matplotlib.pyplot as plt,`” we can use data visualization function by “`plt. function name`”
- Seaborn
 - A library to make Matplotlib's charts more beautiful.
- In the following slides, we will use Matplotlib.

Import Matplot Library



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib as mpl
4 import seaborn as sns
5
6 #Show the chart in that location
7 %matplotlib inline
```

Scatter Plot (1/2)



- A scatter plot (aka scatter chart, scatter graph) uses dots to represent values for two different numeric variables.
- The position of each dot on the horizontal and vertical axis indicates values for an individual data point.
- Scatter plots are used to observe relationships between variables.

Scatter Plot (1/2)

- By “`plt.plot(x,y,'o')`”
can generate a scatter plot.
 - ‘o’ is used to specify the type of dots.

```
# 散佈圖
import numpy.random as random

# 固定種子值
random.seed(0)

# x軸的資料
x = np.random.randn(30)

# y軸的資料
y = np.sin(x) + np.random.randn(30)

# 指定圖形的大小 (可以試著修改20或6)
plt.figure(figsize=(20, 6))

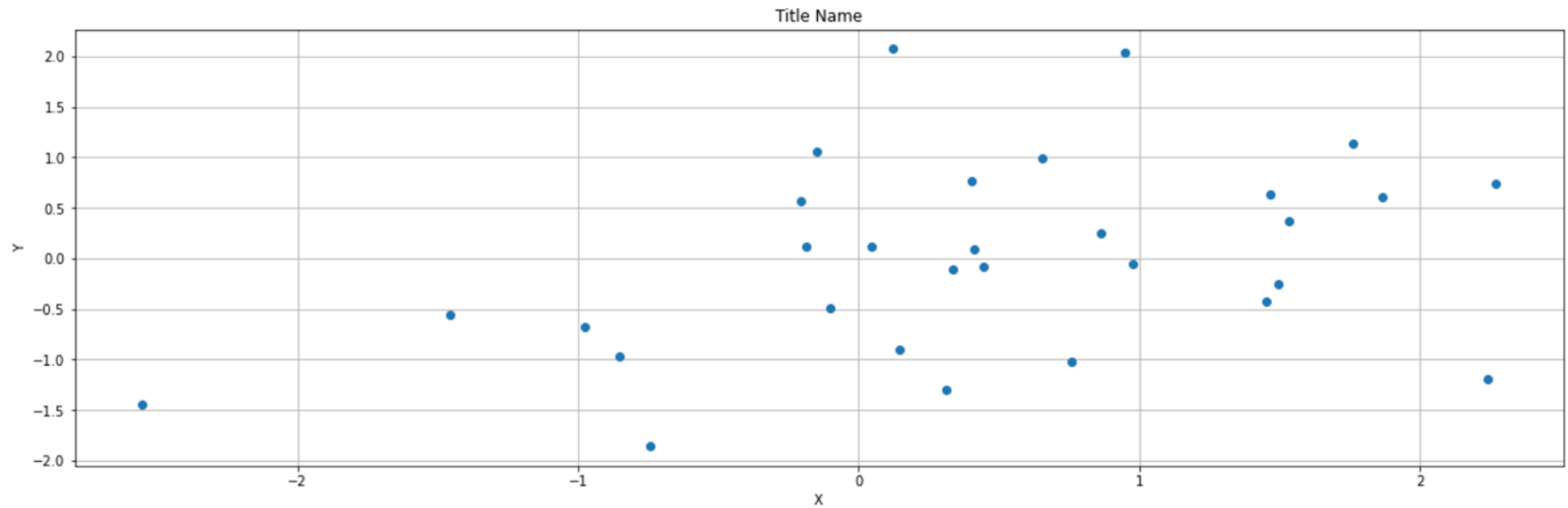
# 描繪圖形
plt.plot(x, y, 'o')

#也能使用scatter描繪散佈圖
#plt.scatter(x, y)

# 標題
plt.title('Title Name')
# x軸標籤
plt.xlabel('X')
# y軸標籤
plt.ylabel('Y')

# 顯示grid (圖形中的格線)
plt.grid(True)
```

Scatter Plot (2/2)



Continuous Scatter Plot (1/2)



- If the data is continuous, the chart will be like curve instead of dot.

連續值散佈圖

```
# 固定種子值
np.random.seed(0)

# 資料範圍
numpy_data_x = np.arange(1000)

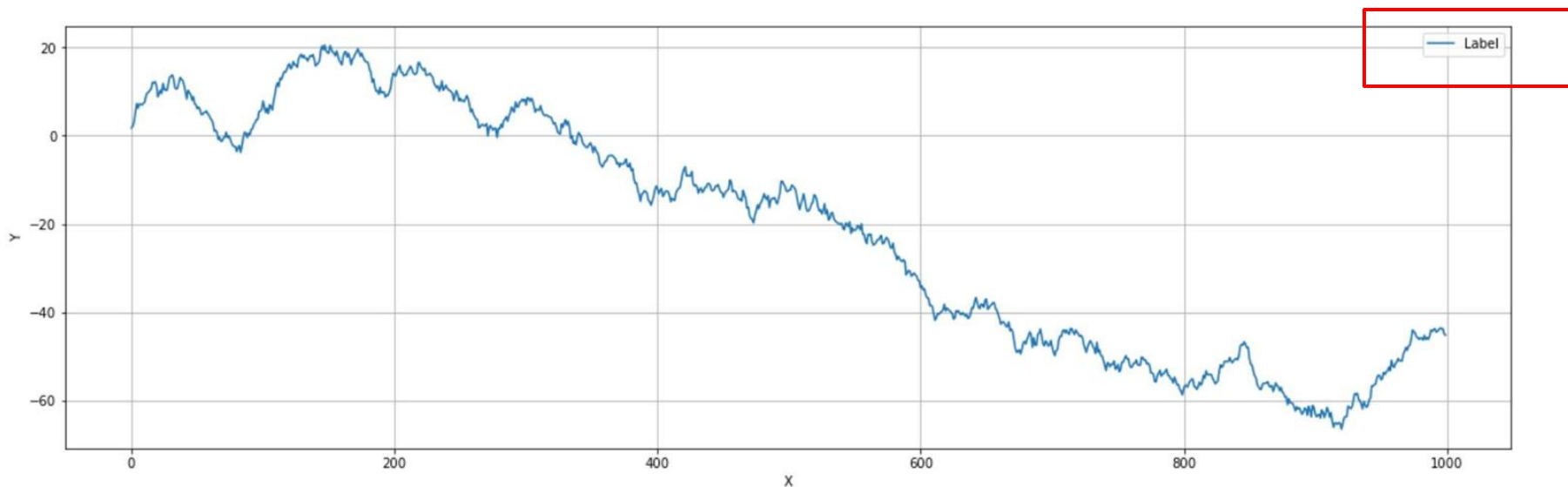
# 生成亂數與累積和
numpy_random_data_y = np.random.randn(1000).cumsum()

# 指定圖形大小
plt.figure(figsize=(20, 6))

# 可藉由label= 與 legend來加上標籤
plt.plot(numpy_data_x, numpy_random_data_y, label='Label')
plt.legend()

plt.xlabel('X')
plt.ylabel('Y')
plt.grid(True)
```

Continuous Scatter Plot(2/2)



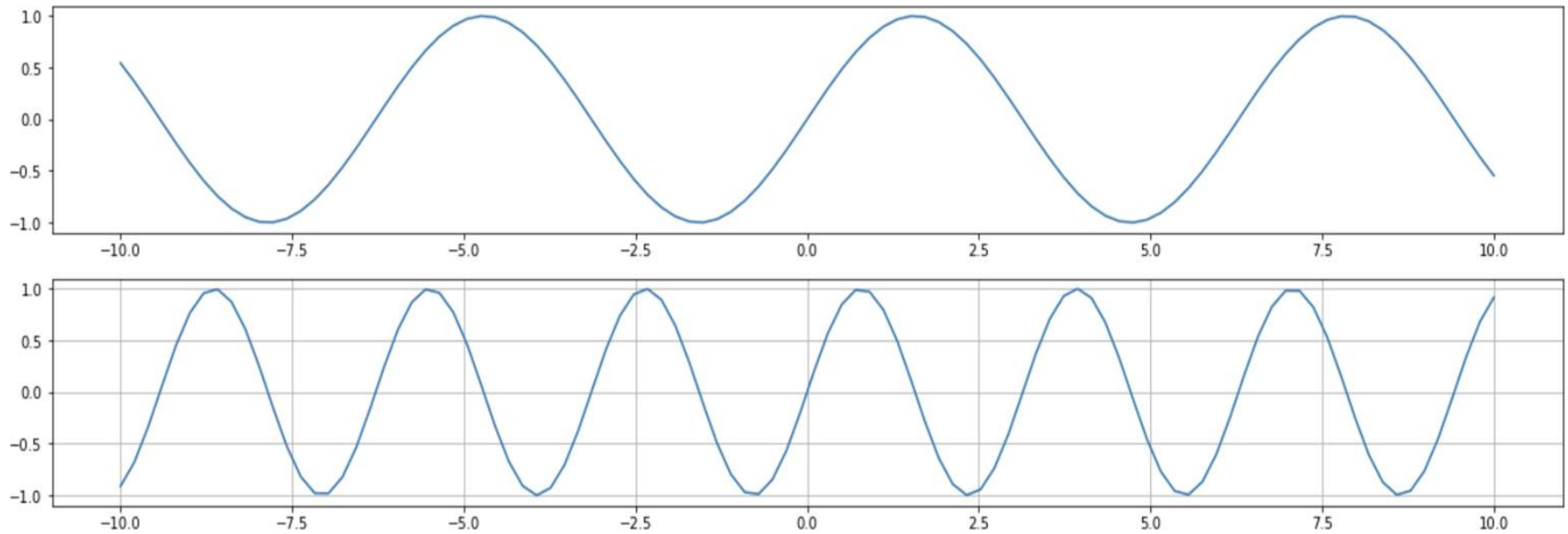
Subplot (1/2)

- We can divide the plots into multiple by “subplot.”
- `plt.subplot(2,1,1)`
 - Represents the plot of second row and the first column.
- `linspace(-10, 10,100)`
 - Divide 100 numbers from -10~10.

圖形分割

```
: plt.figure(figsize=(20, 6))  
# 2列1行的圖形第一個  
plt.subplot(2,1,1)  
# 從-10~10分割出100個數字之串列並取出  
x = np.linspace(-10, 10,100)  
plt.plot(x, np.sin(x))  
  
# 2列1行的圖形第二個  
plt.subplot(2,1,2)  
y = np.linspace(-10, 10,100)  
plt.plot(y, np.sin(2*y))  
  
plt.grid(True)
```

Subplot(2/2)



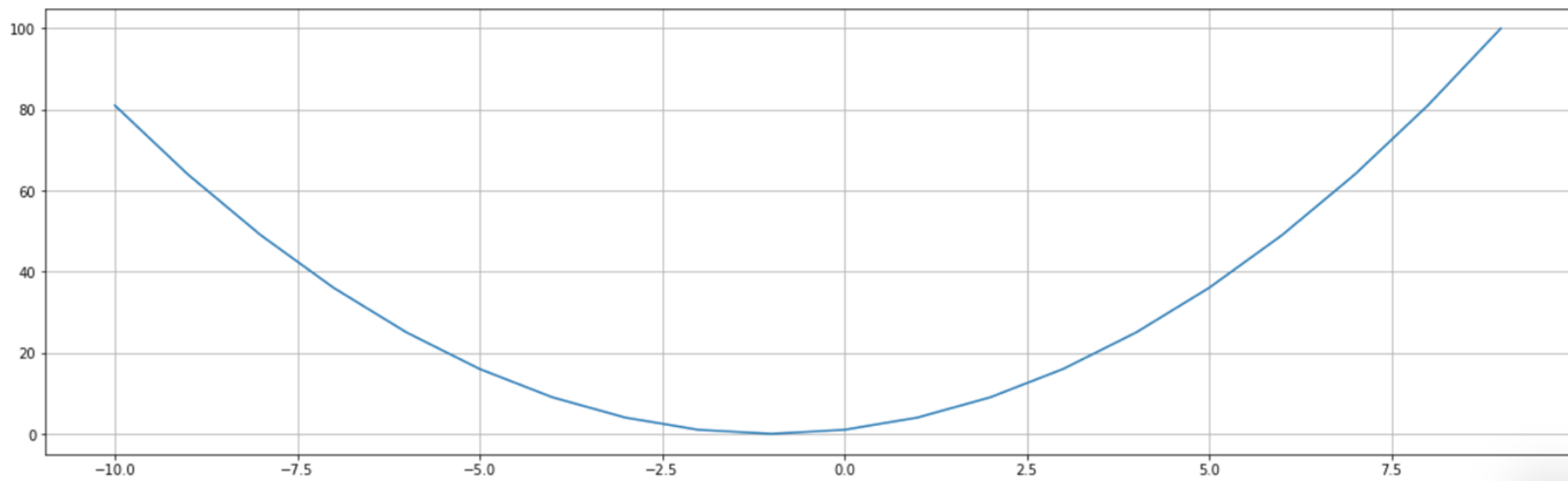


Function Graph (1/2)

- Draw the graph of $f(x)=x^2+2x+1$.

```
def my_function(x):  
    return x ** 2 + 2 * x + 1  
  
x = np.arange(-10, 10)  
plt.figure(figsize = (20, 6))  
plt.plot(x, my_function(x))  
plt.grid(True)
```

Function Graph (2/2)



Histogram (1/3)



- A histogram is an approximate representation of the distribution of numerical data.
- When you want to observe the **all picture of data**, you can use histogram.
- By histogram, we can understand the which value is more, which value is less, and whether there is any deviation.
- We can use **“hist” to generate histogram**.

Histogram (2/3)



直方圖

:

```
random.seed(0)
```

```
# 指定圖形的大小
```

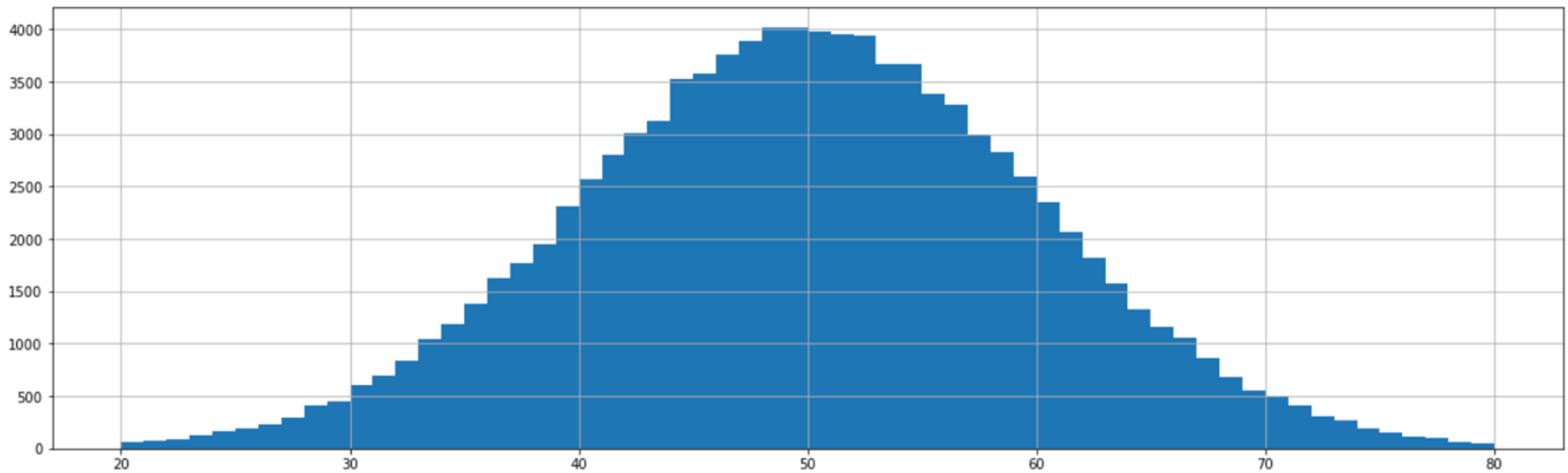
```
plt.figure(figsize = (20, 6))
```

```
# 畫直方圖
```

```
plt.hist(np.random.randn(10 ** 5) * 10 + 50, bins = 60, range = (20, 80))
```

```
plt.grid(True)
```


Histogram (3/3)



Outline of Data Visualization



- What is data visualization?
- Basic library for data visualization.
- **Advanced library data visualization.**
- Stock market data visualization.

Bar Chart (1/3)



- A bar chart provides a way of showing data values represented as vertical bars. It is sometimes used to show trend data, and the comparison of multiple data sets side by side.
 - We can use **bar function in pyplot module**.
 - Show the tags in bar chart, we can use “**xtick**” function.
 - Put the graph in the center: **align='center'**
-
- Q: What is the difference between histogram and bar chart?



Bar Chart(2/3)

```
# 顯示的資料
x = [1, 2, 3]
y = [10, 1, 4]

# 指定圖形的大小
plt.figure(figsize = (10, 6))

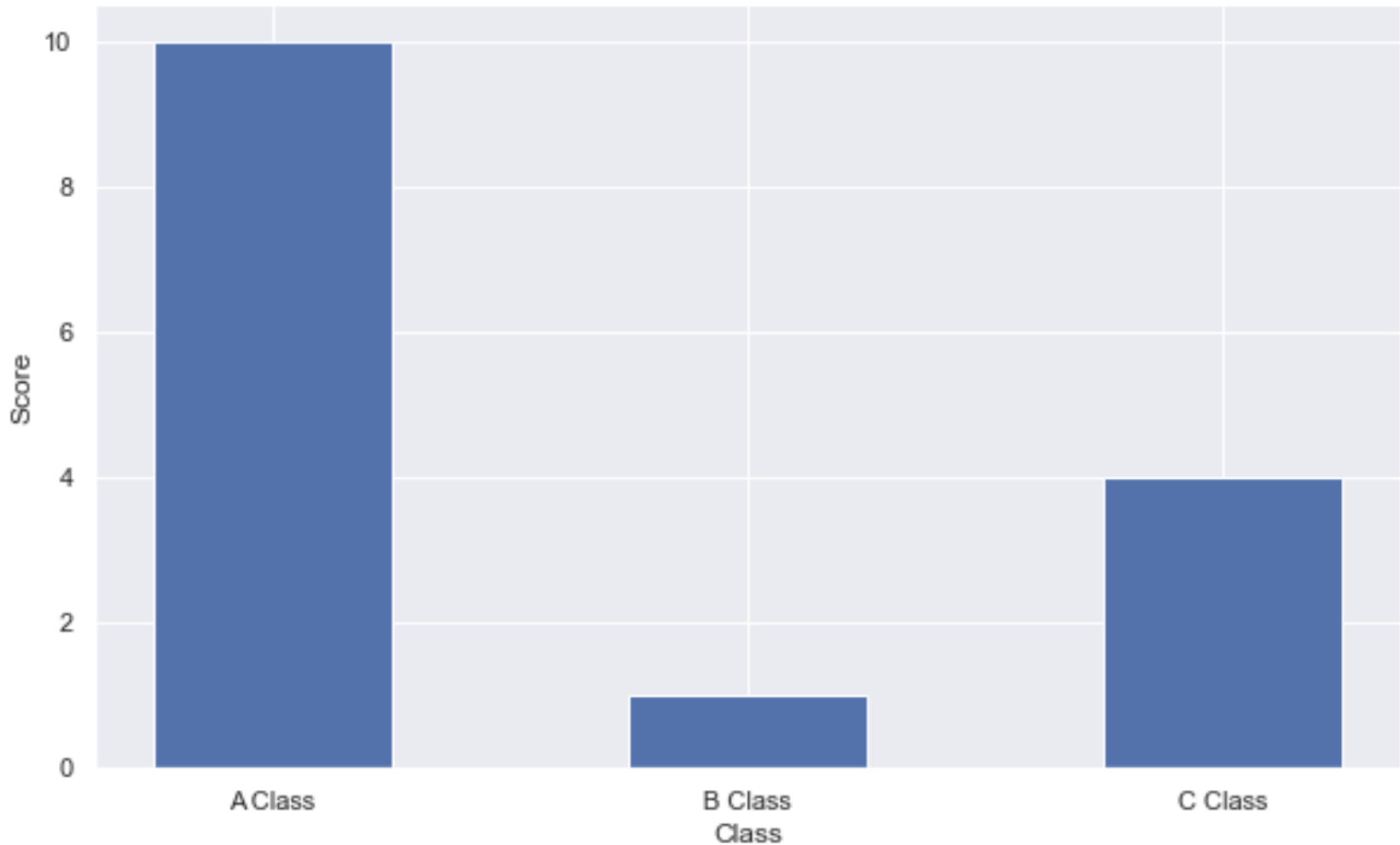
plt.bar(x, y, align='center', width = 0.5)

# 長條圖的各個標籤
plt.xticks(x, ['A Class', 'B Class', 'C Class'])

# 設定x與y的標籤
plt.xlabel('Class')
plt.ylabel('Score')

# 顯示格線
plt.grid(True)
```

Bar Chart (3/3)





Horizontal Bar Chart (1/2)

- We can use “`barh`” to show horizontal bar chart.
- Exchange x axis and y axis and set label again.

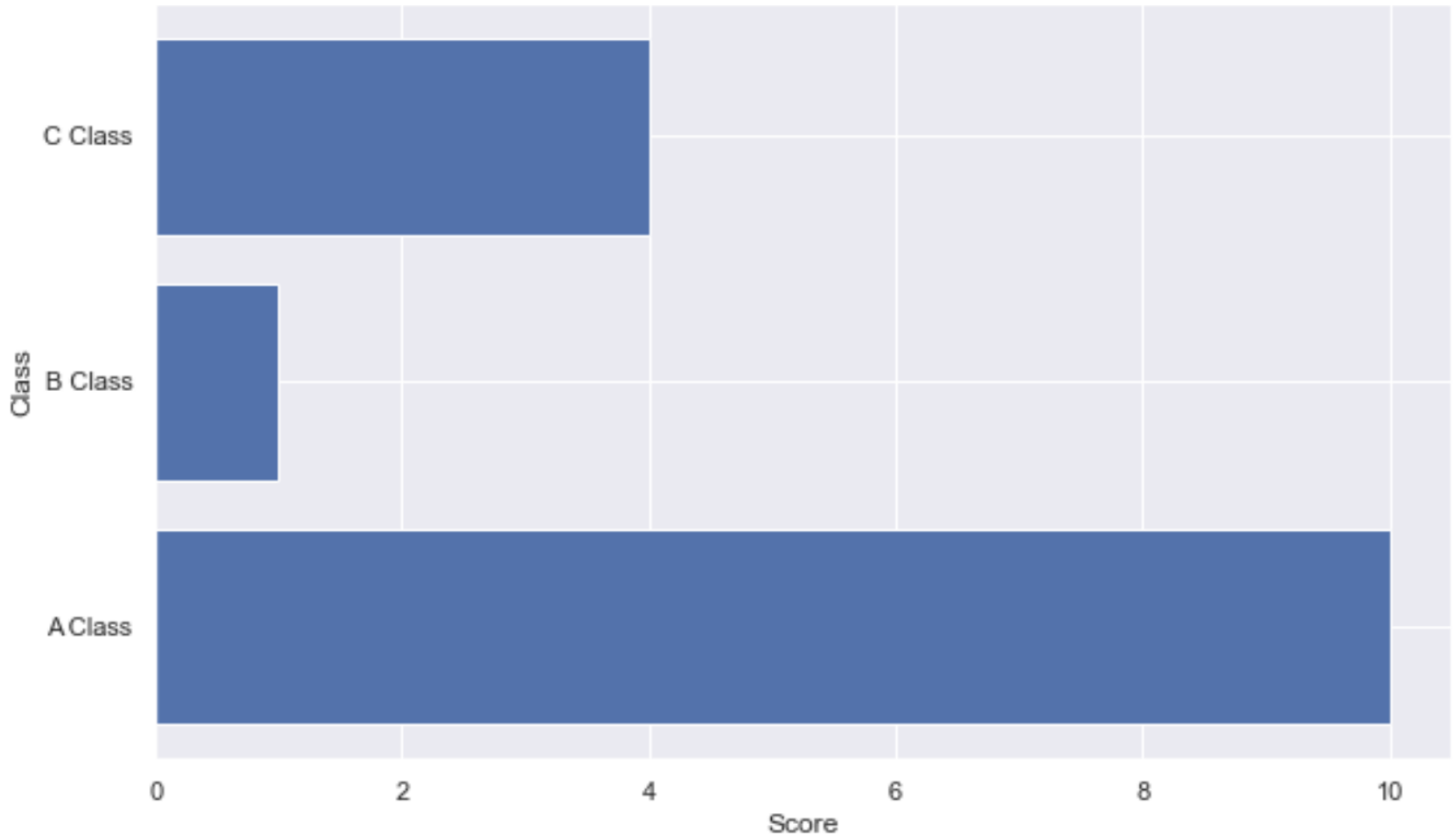
橫條圖

```
# 顯示的資料
x = [1, 2, 3]
y = [10, 1, 4]

# 指定圖形的大小
plt.figure(figsize = (10, 6))

plt.barh(x, y, align = 'center')
plt.yticks(x, ['A Class', 'B Class', 'C Class'])
plt.ylabel('Class')
plt.xlabel('Score')
plt.grid(True)
```

Horizontal Bar Chart (2/2)



Show Many Bar Charts (1/2)



- Visualize first and final math grades by class for comparison.

描繪多個長條圖

```
: #資料的準備
y1 = np.array([30, 10, 40])
y2 = np.array([10, 50, 90])

# x軸的資料
x = np.arange(len(y1))

# 圖形的寬度
w = 0.4

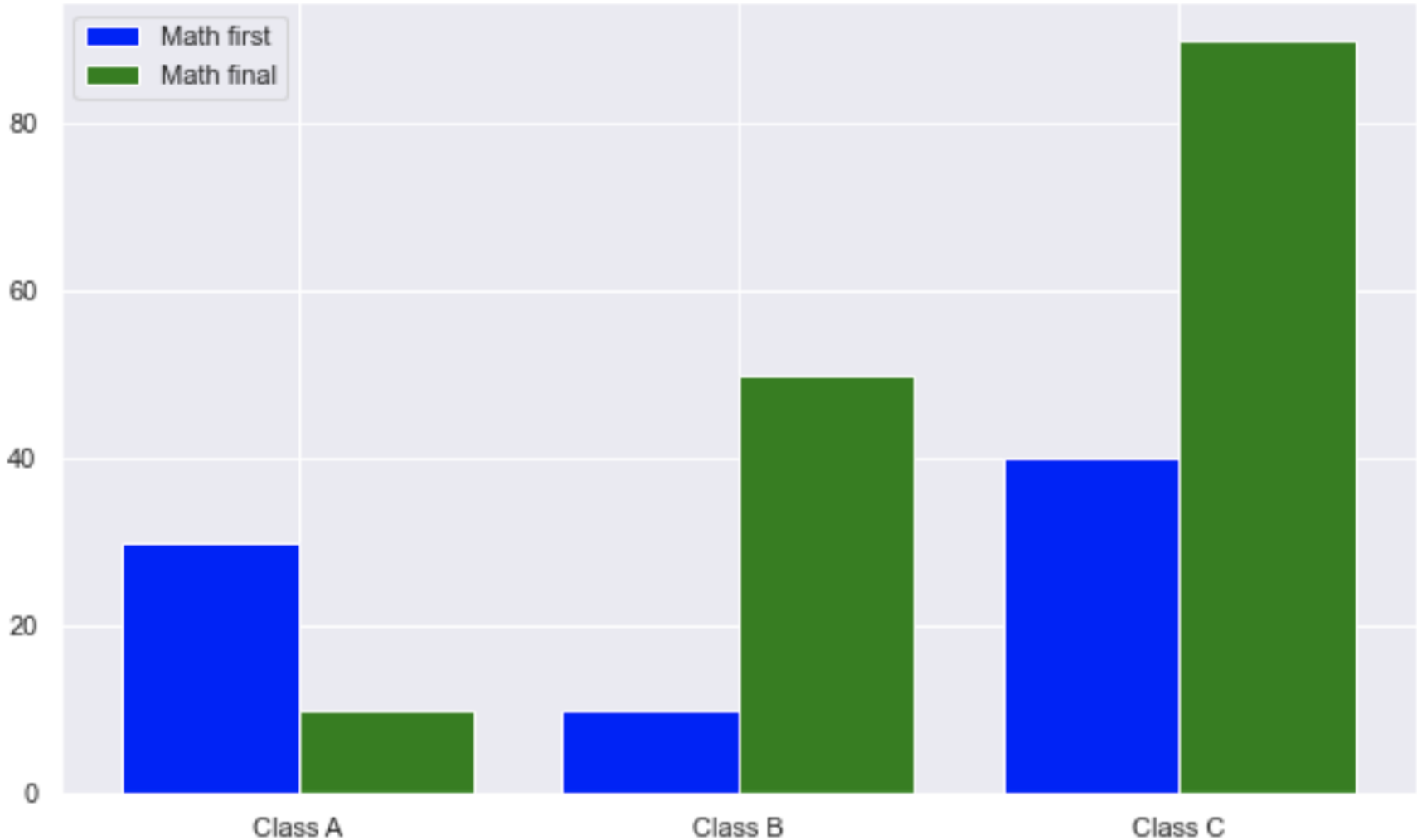
# 指定圖形的大小
plt.figure(figsize = (10, 6))

# 描繪圖形，對於y2，必須要右移一個寬度
plt.bar(x, y1, color = 'blue', width = w, label = 'Math first', align = 'center')
plt.bar(x + w, y2, color='green', width = w, label = 'Math final', align = 'center')

# 讓圖利配置於最適當的位置
plt.legend(loc = 'best')

plt.xticks(x + w / 2, ['Class A', 'Class B', 'Class C'])
plt.grid(True)
```


Show Many Bar Charts (2/2)



Stacked Bar Chart (1/3)



- Same with using “bar” function.
- Do different setting for “bottom parameters.”
- Specify the graphics to be stacked on top in the bar parameter.
 - “bottom= the graphics to be stacked on top”

Stacked Bar Chart (2/3)



堆疊長條圖

資料的準備

```
height1 = np.array([100, 200, 300, 400, 500])  
height2 = np.array([1000, 800, 600, 400, 200])
```

x軸

```
x = np.array([1, 2, 3, 4, 5])
```

指定圖形大小

```
plt.figure(figsize = (10, 6))
```

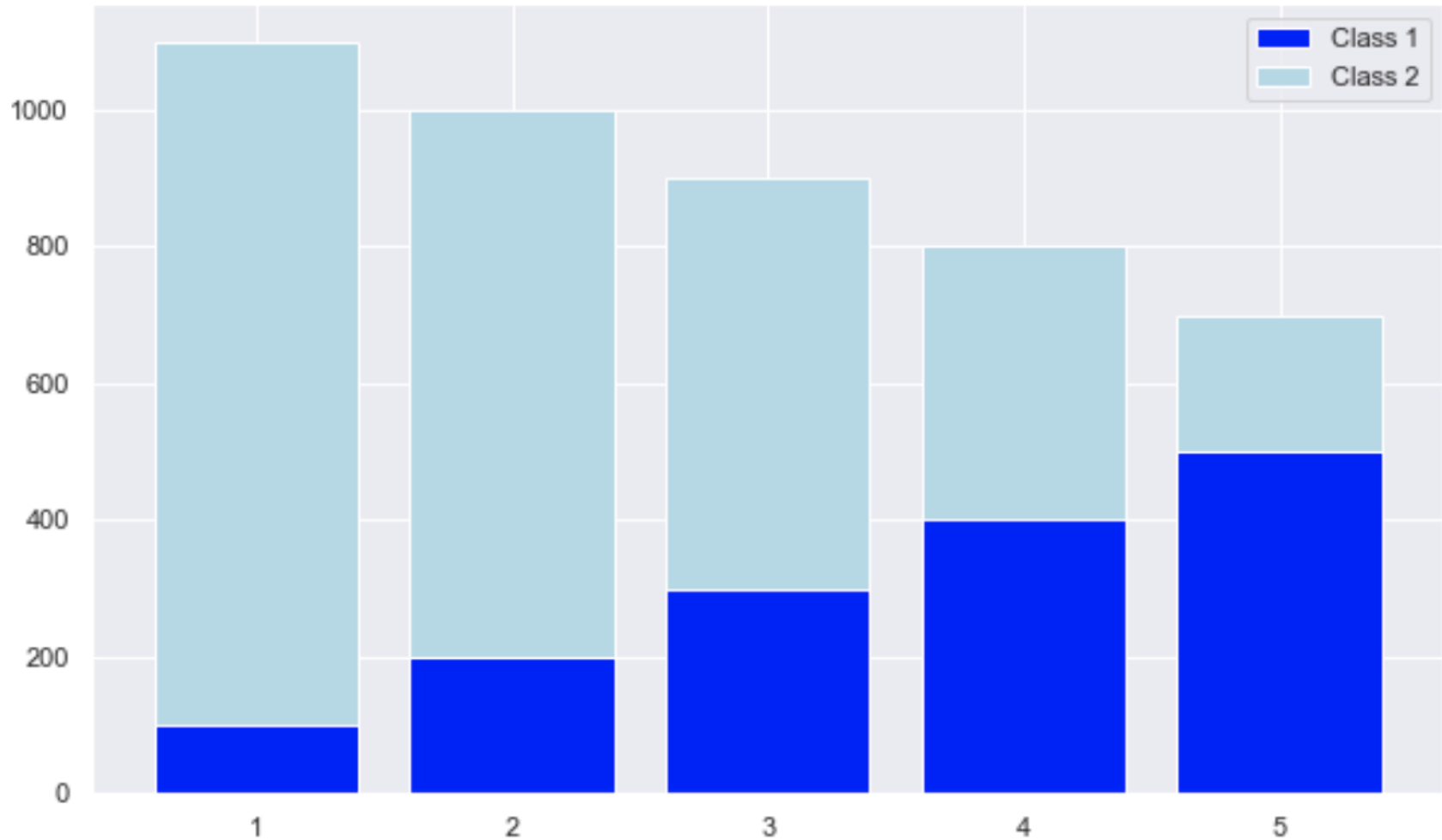
畫堆疊長條圖

```
p1 = plt.bar(x, height1, color = 'blue')  
p2 = plt.bar(x, height2, bottom = height1, color='lightblue')
```

顯示不同顏色代表什麼

```
plt.legend((p1[0], p2[0]), ('Class 1', 'Class 2'))
```

Stacked Bar Chart (3/3)



Pie Chart (1/3)



- A pie chart (or a circle chart) is a circular statistical graphic, which is divided into slices to illustrate numerical proportion.
- In a pie chart, the arc length of each slice (and consequently its central angle and area) is proportional to the quantity it represents.

Pie Chart (2/3)



圓餅圖

```
labels = ['Frogs', 'Hogs', 'Dogs', 'Logs']
sizes = [15, 30, 45, 10]
colors = ['yellowgreen', 'gold', 'lightskyblue', 'lightcoral']
explode = (0, 0.1, 0, 0) 讓第二塊0.1的距離
```

指定圖形大小

```
plt.figure(figsize = (15, 6))
```

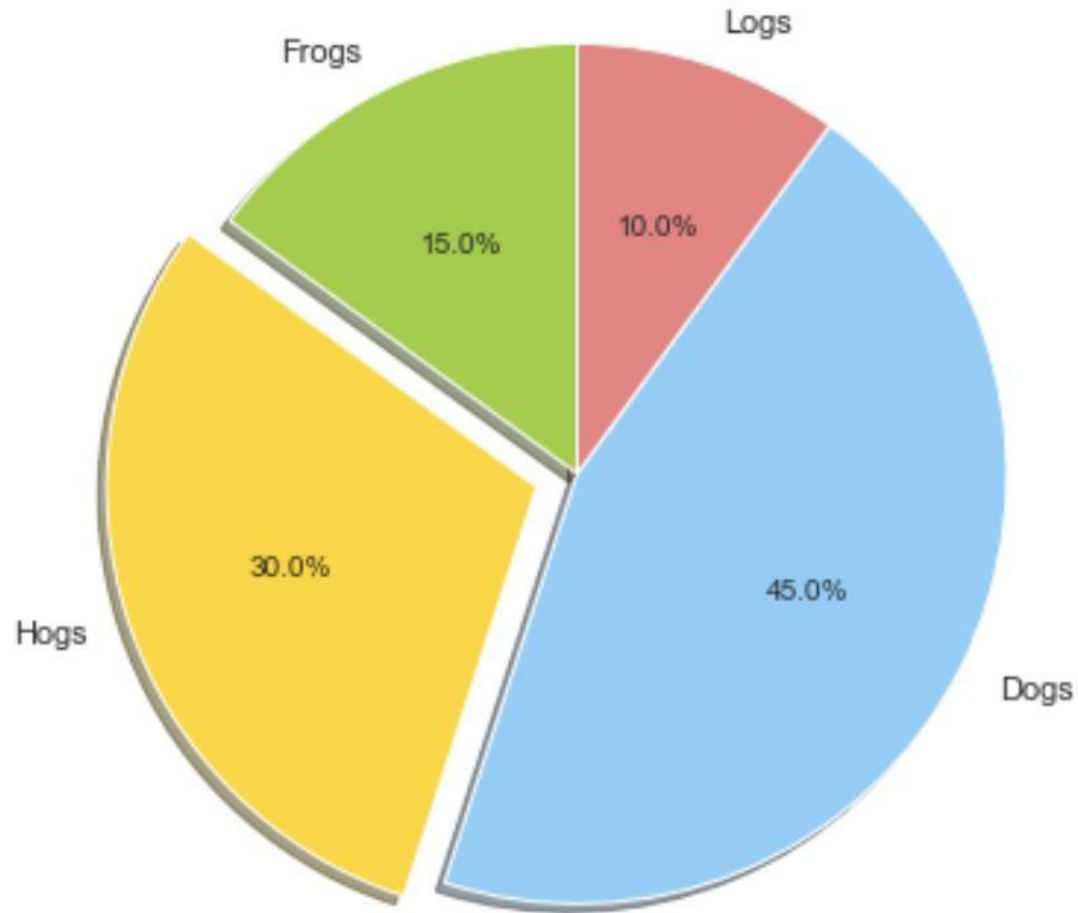
顯示圖型

```
plt.pie(sizes, explode = explode, labels = labels, colors = colors,
        autopct = '%1.1f%%', shadow = True, startangle = 90)
```

維持比例描繪圖形

```
plt.axis('equal')
```

Pie Chart (3/3)



Bubbles Diagram (1/2)



泡泡圖

```
N = 25

# 隨機生成x,y資料
x = np.random.rand(N)
y = np.random.rand(N)

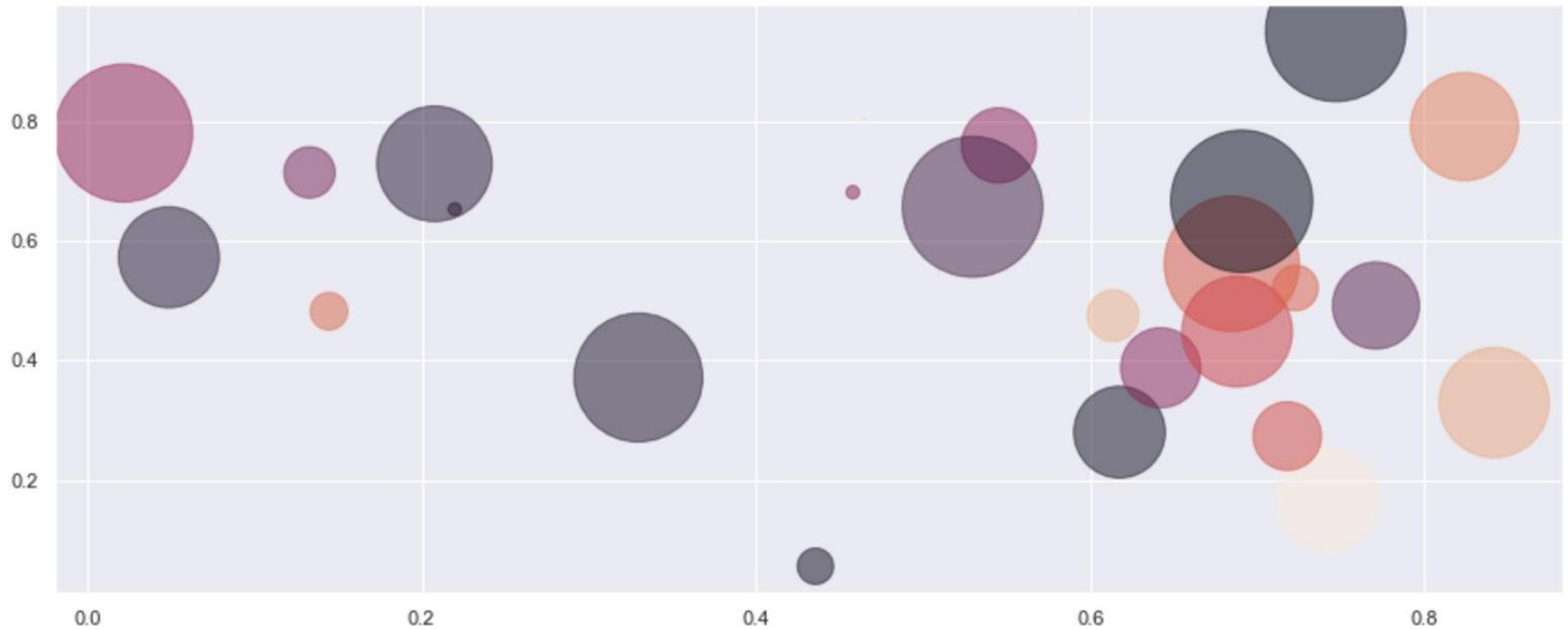
# color編號
colors = np.random.rand(N)

# 讓泡泡的大小顯得散亂
area = 10 * np.pi * (15 * np.random.rand(N)) ** 2

plt.figure(figsize = (15, 6))

# 描繪圖形
plt.scatter(x, y, s = area, c = colors, alpha = 0.5)
plt.grid(True)
```


Bubbles Diagram (2/2)



About Data Visualization



- In recent years, data analysis and data visualization have attracted much attention.
- There are many data visualization tools such as Tableau
、 Excel, PowerBI.
- In companies, they usually use these tools instead of Python.
- However, using Python library is more flexible to adjust the graph. As an engineer, we still need to learn the library for data visualization.

Outline of Data Visualization



- What is data visualization?
- Basic library for data visualization.
- Advanced library data visualization.
- Stock market data visualization.

Appendix : Three Steps for Crawler



- 1) Request the specified URL to get the response
- 2) Parse the response content and analyze the required information from it
- 3) Save information from the previous analysis to a database or file

Requests module: read website files



- To collect information systematically and automatically on the Internet, **you must extract the page content or files on the website for processing.**
- Python provides a "requests" module.
 - Users can easily request the website and get the response content.
- Anaconda has built in
- Query installed packages with this command : **pip list**
- If you use other environments. Use this command : ***pip install -U requests***



send “GET” request

- When the browser is opened, and the URL is sent, the designated web server responds after receiving the request, and the web page can be seen in the browser. The method of this request is called **GET**.
- The requests module can complete GET requests **without going through a browser**. The syntax is:

```
import requests  
Response Object = requests.get(URL)
```

Request object attribute



- The Request object can use the following attributes to get different response content.
- **text:** Get the source data of web page
- The default reading code of Requests is Latin-1. If the code of the read page is different, it will often cause garbled characters. You can set the encoding of the Response object to UTF-8 or Big5
- *Response object.encoding='UTF-8'*
- **content:** Get the binary file data of website
- **status_code:** Get HTTP status code
 - Informational responses, 100–199,
 - Successful responses, 200–299,
 - Redirects, 300–399,
 - Client errors, 400–499,
 - Server errors, 500–599.

Appendix : Three Steps for Crawler



- 1) Request the specified URL to get the response
- 2) Parse the response content and analyze the required information from it
- 3) Save information from the previous analysis to a database or file

BeautifulSoup Module: Web Parsing



- BeautifulSoup: can quickly and accurately analyze and extract specific objects in the page
- Anaconda has built in
- If you use other environments. Use this command :
- **`pip install -U beautifulsoup4`**

The structure of web pages(1/3)



- The content of web pages is plain text, usually saved as [.htm](#) or [.html](#) files.
- A web page uses HTML (Hypertext Markup Language) syntax to construct content [with tags](#) so the browser can show the web page according to its description.

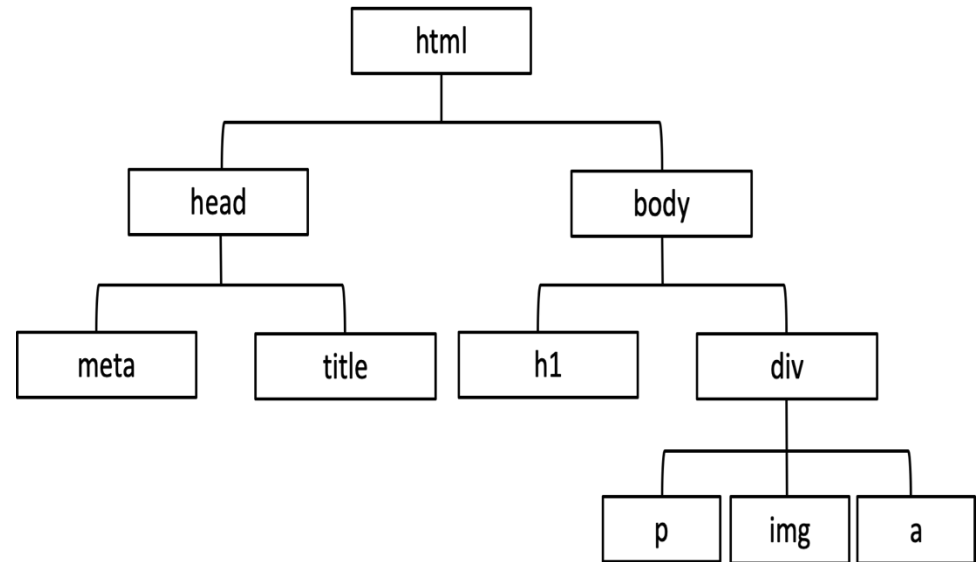
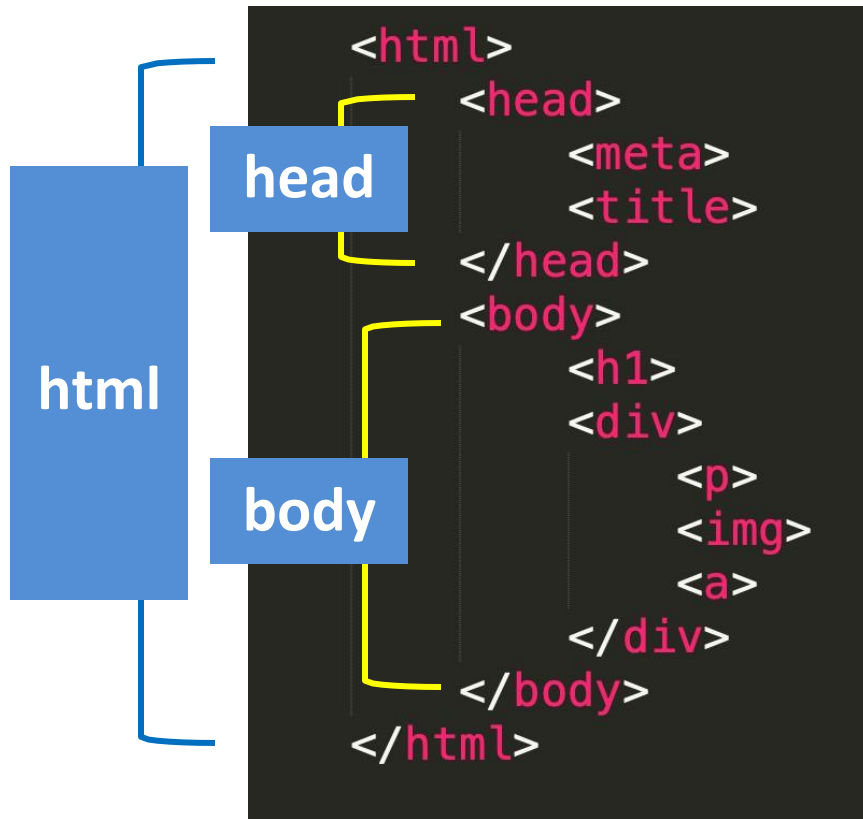
The structure of web pages(2/3)



- HTML provides a structured representation of documents: DOM (Document Object Model, document object model)
- All tags are enclosed by `<.....>`, most have start and end tags
 - `<h1>Title</h1>`
 - `<div>Block Content</div>`
 - `<p>Paragraph</p>`
 - `<img>Image</img>`
 - `<a>Hyperlink</a>`

```
1 <!doctype html>
2 <html>
3   <head>
4     <meta charset="UTF-8">
5     <title>我是網頁標題</title>
6   </head>
7   <body>
8     <h1 class="larger">我是標題</h1>
9     <div>
10      <p>我是段落</p>
11      
12      <a href="http://www.e-happy.com.tw">我是超連結</a>
13    </div>
14  </body>
15 </html>
16
```

The structure of web pages(3/3)



- The function of the BeautifulSoup module parses the web page source code into structured objects, allowing the program to obtain the content quickly.

How to use BeautifulSoup



- After importing BeautifulSoup, use the “requests” module to obtain the source code of the webpage, and then use “`lxml`” to parse the source code.

```
from bs4 import BeautifulSoup
BeautifulSoup Object = BeautifulSoup(Source code, 'lxml')
```

- Creating BeautifulSoup requires two parameters:
- First parameter : The source code to be parsed
- Second parameter : The parser
 - “html.parser” is Python's built-in parser
 - “lxml” is a C-based parser that performs faster

Attributes of BeautifulSoup



Attribute	Description
label name	Returns the specified content ex: sp.title returns the tag content of <title>
text	Returns the text content of the web page after removing all HTML tags

BeautifulSoup: find(), find_all()



Function	Description
find()	Find the first matching tag, return it as a string, Ex:sp.find("a")
find_all()	Find the all matching tag, return it as a list, Ex:sp.find("a")

- Add tag attributes as search criteria

```
BeautifulSoup Object.find() or find_all(Label name, Attribute name=Attribute content)
```

BeautifulSoup: select()



Function	Description
select()	Find the content of the specified CSS selector such as id or class, and return it in a list Ex: Read by id : <code>sp.select("#id")</code> Read by class : <code>sp.select(".classname")</code>

Test



- When do we need data

(1)

- What is the library for data

(2)